Bases de Datos · 202610-2497

Proyecto de Investigación y Desarrollo (I+D)

Bases de datos no relacionales: bases de datos orientadas a grafos

Diseño e implementación de una aplicación basada en base de datos de grafos para la gestión y análisis de relaciones en una red profesional

UniRed: red profesional sobre Neo4j

Integrantes

Mateo Gómez
Valerie Angulo
Sahory Blanco
Sebastián Corbacho
Cristian Viloria

Docente: Wilson Estrada Bernal

Fecha de entrega: 10 de noviembre de 2026

Universidad del Norte · Barranquilla

Contenido

1. Introducción y objetivos 4
1.1 Cumplimiento del enunciado 4
2. Bases de datos de grafos 5
2.1 Concepto 5
2.2 Adyacencia sin índices 5
2.3 Cypher 5
2.4 Tecnología elegida 5
3. Modelo de datos 6
3.1 Nodos 6
3.2 Relaciones 7
3.3 Decisiones de diseño 7
4. Scripts de la base de datos 8
4.1 Esquema 8
4.2 Carga de datos (fragmentos) 9
5. Consultas 9
5.1 CRUD y relaciones 9
5.2 ¿Qué usuarios tienen habilidades similares? 9
5.3 ¿Qué contactos en común existen entre dos profesionales? 10
5.4 ¿Cuál es la ruta más corta entre un usuario y una empresa? 10
5.5 ¿Qué candidatos podrían recomendarse para una oferta? 11
5.6 ¿Qué proyectos comparten tecnologías o miembros? 12
5.7 ¿Qué habilidades son las más demandadas? 12
5.8 Recomendaciones 12
5.9 Analítica 13
5.10 Consultas de la capa social 14
6. Aplicación UniRed 15
6.1 Arquitectura 15
6.2 Inicio de sesión y registro 15
6.3 Feed, publicaciones y reacciones 16
6.4 Perfil, estudios y «Cómo llegas a…» 17
6.5 Mi red, empleos y mensajes 18
6.6 Panel de administración 20
6.7 «Ver Cypher» 21

6.8 Seguridad 22
6.9 Versión móvil 22
6.10 Puesta en marcha y publicación 23
6.11 Ver la base de datos en Neo4j Browser 24
7. Componente de investigación: grafos frente a relacional 25
7.1 El problema de los JOIN 25
7.2 Cuándo el modelo de grafos supera al relacional 26
7.3 Cuándo no conviene (o el relacional es mejor) 26
7.4 Aplicado a este proyecto 26
7.5 Comparación breve 27
8. Guion de la demostración (15 min) 27
9. Conclusiones 28
Referencias 28
Anexo A. 01_schema.cypher 29
Anexo B. 02_seed.cypher 30
Anexo C. 03_queries.cypher 34
Anexo D. 04_red_social.cypher 38

Para actualizar la tabla de contenido en Word: clic derecho sobre ella → **Actualizar campos**.

1. Introducción y objetivos

Muchas aplicaciones actuales manejan información altamente interconectada, donde las relaciones entre los datos son tan importantes como los datos mismos. Las redes profesionales (contactos, empleos, habilidades, proyectos) son un ejemplo típico. Este proyecto diseña e implementa una red profesional sobre **Neo4j**, una base de datos de grafos, y documenta el modelo, los scripts, las consultas y el análisis de cuándo conviene (y cuándo no) este paradigma frente al relacional.

El resultado es **UniRed**, una aplicación web con dos caras sobre el mismo grafo: un **panel de administración** que cubre punto por punto lo que pide el enunciado (CRUD, relaciones, consultas, recomendaciones y analítica) y una **red social** con inicio de sesión, publicaciones, reacciones, comentarios, mensajes y estudios, que muestra el modelo de grafos funcionando como lo usaría una persona real.

Objetivos.

- Afianzar los conocimientos de diseño y modelamiento de bases de datos no relacionales.
- Demostrar la utilidad de las bases de datos de grafos en escenarios con relaciones múltiples y dinámicas, evidenciando sus ventajas frente a modelos tradicionales en análisis de redes y recomendación.

1.1 Cumplimiento del enunciado

Requisito del enunciado	Dónde se cumple
CRUD de usuarios con información personal y profesional	Panel › Usuarios (crear, ver, editar, eliminar con `DETACH DELETE`); registro y edición de perfil en UniRed
Relacionar usuarios con conexiones profesionales	Panel › Relaciones (`CONECTA_CON`); en UniRed, invitaciones `SOLICITA_CONEXION` que al aceptarse crean `CONECTA_CON`
Vincular usuarios con habilidades, empresas y proyectos	Panel › Relaciones y Editar perfil (`TIENE_HABILIDAD`, `TRABAJA_EN`, `PARTICIPA_EN`)
Registrar empresas y sus ofertas	Panel › Empresas y Ofertas; en UniRed, quien trabaja en una empresa publica sus ofertas (`PUBLICA`)
Relacionar ofertas con habilidades requeridas	Al publicar una oferta se crean las relaciones `REQUIERE`
Postularse a una oferta	UniRed › Empleos y Panel › Ofertas (`POSTULA_A` con estado)
Consultar caminos y conexiones	Panel › Consultas (ruta más corta dibujada); perfil «Cómo llegas a…» con `shortestPath`
Recomendaciones de contactos, empleos y colaboradores	Panel › Recomendaciones; UniRed › «Personas que quizá conozcas» y «Empleos para ti»
Panel de analítica: influyentes, comunidades, habilidades demandadas	Panel › Analítica (grado + PageRank, comunidades, demanda y brecha)
Las 6 preguntas del enunciado	Panel › Consultas del enunciado y `03_queries.cypher` (C1–C6)

Requisito del enunciado	Dónde se cumple
Documento, scripts y componente de investigación	Este documento: secciones 2–8 y anexos A–C

2. Bases de datos de grafos

2.1 Concepto

Una base de datos de grafos almacena la información como **nodos** (entidades) y **relaciones** (conexiones entre entidades). Ambos pueden tener **propiedades** en forma de pares clave-valor. Neo4j implementa el modelo de grafo de propiedades etiquetadas (labeled property graph):

- **Nodo:** entidad con una o más etiquetas (p. ej. `:Usuario`) y propiedades.
- **Relación:** conexión dirigida, con un tipo obligatorio (p. ej. `:CONECTA_CON`), un nodo origen, un nodo destino y propiedades opcionales.
- **Propiedad:** dato clave-valor asociado a nodos o relaciones.

2.2 Adyacencia sin índices

La característica diferencial de motores como Neo4j es la **adyacencia sin índice** (index-free adjacency): cada nodo guarda referencias directas a sus relaciones, de modo que recorrer una conexión cuesta lo mismo sin importar el tamaño total del grafo. El costo de una consulta depende de la porción del grafo que se recorre, no del volumen total de datos. En el modelo relacional, en cambio, cada salto equivale a un JOIN que consulta índices o recorre tablas.

2.3 Cypher

Cypher es el lenguaje declarativo de Neo4j. Describe patrones con una sintaxis de «arte ASCII»: los nodos van entre paréntesis y las relaciones entre corchetes, por ejemplo `(a:Usuario)-[:CONECTA_CON]->(b:Usuario)`. Soporta recorridos de longitud variable (`[*..8]`), funciones de camino como `shortestPath`, agregaciones, comprensión de listas y subconsultas `COUNT { }` / `COLLECT { }`.

2.4 Tecnología elegida

Componente	Elección
Motor de grafos	Neo4j 5.26 Community (portable en el PC, sin instalador; también funciona con Neo4j Desktop o AuraDB)
Lenguaje de consulta	Cypher
Aplicación	Python 3 + Flask (plantillas Jinja2, Bootstrap 5) con el controlador oficial `neo4j`
Analítica	Cypher + networkx (PageRank, comunidades); alternativa opcional: Neo4j GDS
Visualización	vis-network (grafos interactivos) y Chart.js (gráficas)

Componente	Elección
Servidor y publicación	waitress (servidor WSGI) + Cloudflare Tunnel (enlace https público desde el PC)
Seguridad	Contraseñas con hash (werkzeug/PBKDF2), token CSRF, consultas parametrizadas, consola de solo lectura

3. Modelo de datos

El modelo parte de las entidades y relaciones sugeridas en el enunciado. Se añadió `USA_TECNOLOGIA` (Proyecto → Habilidad), necesaria para responder «¿qué proyectos comparten tecnologías?», y una **capa social** (publicaciones, reacciones, comentarios, invitaciones, estudios y mensajes) que convierte la red en una aplicación usable sin alterar el núcleo pedido.

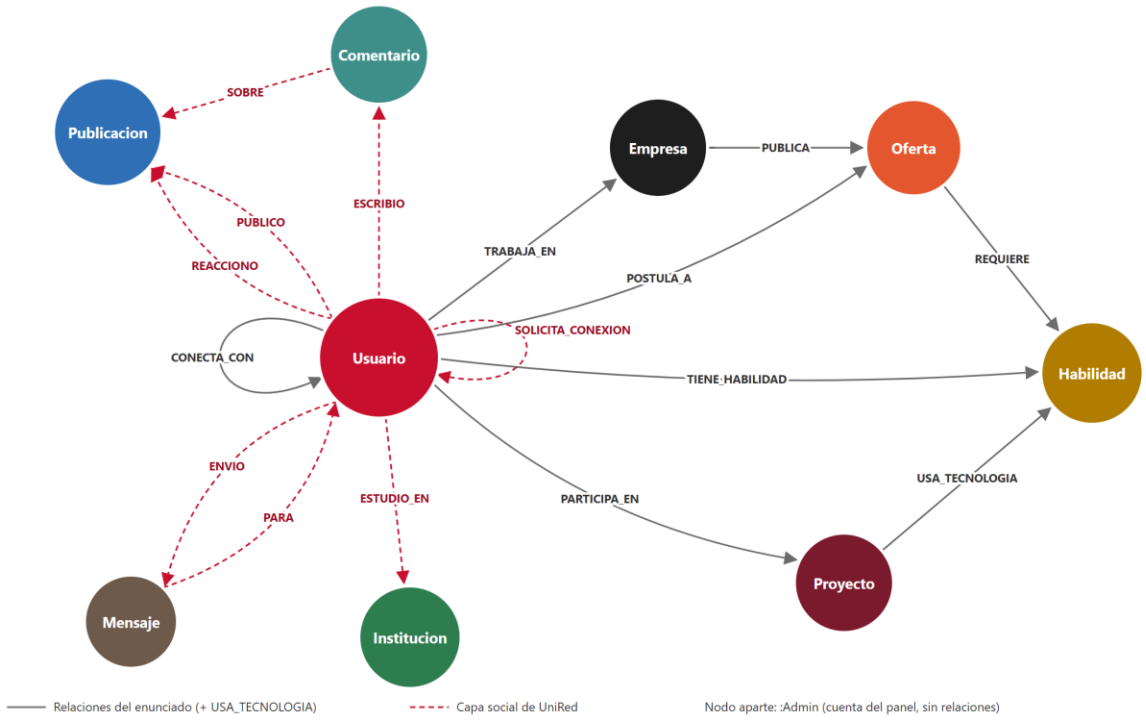


Figura 1. *Modelo del grafo de UniRed. En gris, las relaciones del enunciado; en rojo punteado, la capa social.*

3.1 Nodos

Etiqueta	Descripción	Propiedades
Usuario	Persona registrada	id (único), nombre, email (único), clave_hash, titular, ciudad, anios_experiencia, acerca, foto, portada
Empresa	Organización o compañía	id (único), nombre, sector, ciudad
Habilidad	Competencia o conocimiento	nombre (único)
Proyecto	Trabajo o iniciativa	id (único), nombre, descripcion

Etiqueta	Descripción	Propiedades
Oferta	Vacante laboral	id (único), titulo, ubicacion, salario, estado (ABIERTA/CERRADA)
Institucion	Universidad o centro de estudios	nombre (único)
Publicacion	Publicación del feed	id (único), texto, media, tipo_media (imagen/video), fecha
Comentario	Comentario a una publicación	id (único), texto, fecha
Mensaje	Mensaje privado entre contactos (dura 24 h)	id (único), texto, fecha, leido
Admin	Cuenta del panel de administración	id, email (único), nombre, clave_hash

3.2 Relaciones

Relación	Dirección	Propiedades
CONECTA_CON	Usuario → Usuario	desde (fecha). Se consulta sin dirección
TRABAJA_EN	Usuario → Empresa	cargo, desde
TIENE_HABILIDAD	Usuario → Habilidad	nivel (1 a 5)
PARTICIPA_EN	Usuario → Proyecto	—
PUBLICA	Empresa → Oferta	—
REQUIERE	Oferta → Habilidad	—
POSTULA_A	Usuario → Oferta	fecha, estado (EN_REVISION, ENTREVISTA, SELECCIONADO, DESCARTADO)
USA_TECNOLOGIA	Proyecto → Habilidad	—
SOLICITA_CONEXION	Usuario → Usuario	fecha. Al aceptarse se borra y se crea CONECTA_CON
ESTUDIO_EN	Usuario → Institucion	id, titulo, area, inicio, fin
PUBLICO	Usuario → Publicacion	—
REACCIONO	Usuario → Publicacion	tipo (recomendar, celebrar, apoyar, encanta, interesante, divertido), fecha
ESCRIBIO / SOBRE	Usuario → Comentario → Publicacion	—
ENVIO / PARA	Usuario → Mensaje → Usuario	—

3.3 Decisiones de diseño

- **Habilidad como nodo, no como propiedad.** Al ser nodo, conecta usuarios, ofertas y proyectos; permite descubrir similitudes y demanda recorriendo el grafo en lugar de comparar listas de texto.
- **Los datos que pertenecen a la relación van en la relación.** El nivel de dominio vive en `TIENE_HABILIDAD`, el estado de la postulación en `POSTULA_A` y el tipo de reacción en `REACCIONO`.

- **CONECTA_CON se guarda en un solo sentido y se consulta sin dirección.** Evita duplicar aristas; las consultas usan `-[:CONECTA_CON]-`.
- **La invitación es una relación distinta.** `SOLICITA_CONEXION` representa una conexión que todavía no existe. Por eso las consultas de caminos del enunciado solo recorren las relaciones profesionales: una invitación pendiente no es un camino real.
- **Institución como nodo.** «Estudió en la misma universidad» se vuelve un patrón de dos saltos que alimenta las recomendaciones de contactos.
- **Las notificaciones no se guardan.** Se calculan con una consulta `UNION ALL` sobre las reacciones, comentarios e invitaciones que apuntan al usuario, así no hay datos duplicados que mantener.
- **Los mensajes vencen a las 24 horas.** Un proceso de la app ejecuta cada 10 minutos `MATCH (m:Mensaje) WHERE m.fecha < datetime() - duration({hours: 24}) DETACH DELETE m`, para no llenar la base.
- **Admin separado de Usuario.** La cuenta del panel tiene su propia etiqueta, así no aparece en recomendaciones, analítica ni caminos.
- **Restricciones de unicidad.** Cada nodo tiene una clave única (y `email` en Usuario); además crean índices que aceleran los `MATCH` por id. Hay índices en `Oferta.estado`, `Publicacion.fecha` y `Mensaje.fecha`.

4. Scripts de la base de datos

Los scripts están en la carpeta `scripts/` y se ejecutan en orden con `python cargar_datos.py` (o pegándolos en Neo4j Browser). Los datos se cargan con `MERGE`, de modo que ejecutarlos varias veces no duplica información. El contenido completo está en los **anexos A, B y C**.

Archivo	Contenido
01_schema.cypher	Restricciones de unicidad e índices (incluye los nodos de la capa social)
02_seed.cypher	Datos de ejemplo: 20 usuarios, 6 empresas, 16 habilidades, 8 proyectos, 10 ofertas, conexiones, habilidades, proyectos y postulaciones
03_queries.cypher	CRUD, creación de relaciones, preguntas del enunciado, recomendaciones y analítica
04_red_social.cypher	7 instituciones y estudios, 10 publicaciones, reacciones, comentarios, invitaciones y mensajes recientes

4.1 Esquema

```
CREATE CONSTRAINT usuario_id   IF NOT EXISTS FOR (u:Usuario)   REQUIRE u.id IS UNIQUE;
CREATE CONSTRAINT usuario_mail IF NOT EXISTS FOR (u:Usuario)   REQUIRE u.email IS UNIQUE;
CREATE CONSTRAINT habilidad_nm IF NOT EXISTS FOR (h:Habilidad) REQUIRE h.nombre IS UNIQUE;
// ... empresa, proyecto, oferta, publicacion, comentario, mensaje, institucion y admin
CREATE INDEX oferta_estado     IF NOT EXISTS FOR (o:Oferta)      ON (o.estado);
CREATE INDEX mensaje_fecha     IF NOT EXISTS FOR (m:Mensaje)     ON (m.fecha);
```

4.2 Carga de datos (fragmentos)

```
UNWIND [
  {id:'u1', nombre:'Ana Torres', email:'ana@mail.com',
   titular:'Ingeniera de Datos', ciudad:'Bogotá', anios:6}, ...
] AS r
MERGE (u:Usuario {id:r.id})
SET u.nombre=r.nombre, u.email=r.email, u.titular=r.titular,
    u.ciudad=r.ciudad, u.anios_experiencia=r.anios;

// Estudios: (Usuario)-[:ESTUDIO_EN]->(Institucion)
UNWIND [['u16','Universidad del Norte','Ingeniería de Sistemas','Bases de datos','2020','2025'],
...] AS r
MATCH (u:Usuario {id:r[0]}), (i:Institucion {nombre:r[1]})
MERGE (u)-[s:ESTUDIO_EN {id: 's-' + r[0] + '-' + r[4]}]->(i)
  ON CREATE SET s.titulo = r[2], s.area = r[3], s.inicio = r[4], s.fin = r[5];
```

Las contraseñas no se escriben en los scripts: `cargar_datos.py` asigna a los usuarios de ejemplo un hash de la clave de demostración y crea la cuenta de administrador con la clave definida en el archivo `.env`.

5. Consultas

Las siguientes consultas están en `03_queries.cypher` y se invocan desde la aplicación con parámetros (`$id`, `$o`…). En la app, el botón **Ver Cypher** muestra la consulta exacta y sus parámetros en cada pantalla (figura 12).

5.1 CRUD y relaciones

```
CREATE (u:Usuario {id:'u21', nombre:'Elena Prado', email:'elena@mail.com',
                   titular:'Ingeniera de Datos', ciudad:'Bogotá', anios_experiencia:3})
RETURN u;

MATCH (u:Usuario {id:'u21'})
SET u.titular = 'Científica de Datos Senior', u.anios_experiencia = 4
RETURN u;

MATCH (u:Usuario {id:'u21'})
DETACH DELETE u;
```

`DETACH DELETE` elimina el nodo junto con todas sus relaciones; sin `DETACH`, Neo4j rechaza borrar un nodo que aún tiene conexiones. En la app, antes se borran los nodos que solo existen por ese usuario (sus publicaciones con sus comentarios, sus comentarios y sus mensajes).

```
MATCH (u:Usuario {id:'u4'}), (o:Oferta {id:'o9'})
MERGE (u)-[p:POSTULA_A]->(o)
  ON CREATE SET p.fecha = date(), p.estado = 'EN_REVISION'
RETURN u.nombre, o.titulo, p.estado;
```

5.2 ¿Qué usuarios tienen habilidades similares?

```
MATCH (a:Usuario {id:'u1'})-[:TIENE_HABILIDAD]->(h:Habilidad)<-[:TIENE_HABILIDAD]-(b:Usuario)
WHERE a <> b
WITH a, b, collect(h.nombre) AS compartidas
MATCH (a)-[:TIENE_HABILIDAD]->(ha:Habilidad)
WITH a, b, compartidas, collect(ha) AS habA
MATCH (b)-[:TIENE_HABILIDAD]->(hb:Habilidad)
WITH a, b, compartidas, habA, collect(hb) AS habB
```

```
RETURN b.nombre AS usuario, compartidas,
       size(compartidas) * 1.0 / (size(habA) + size(habB) - size(compartidas)) AS similitud
ORDER BY similitud DESC, usuario LIMIT 10;
```

Se recorre Usuario → Habilidad ← Usuario para hallar las habilidades compartidas y se calcula el **índice de Jaccard** (compartidas / unión de habilidades de ambos).

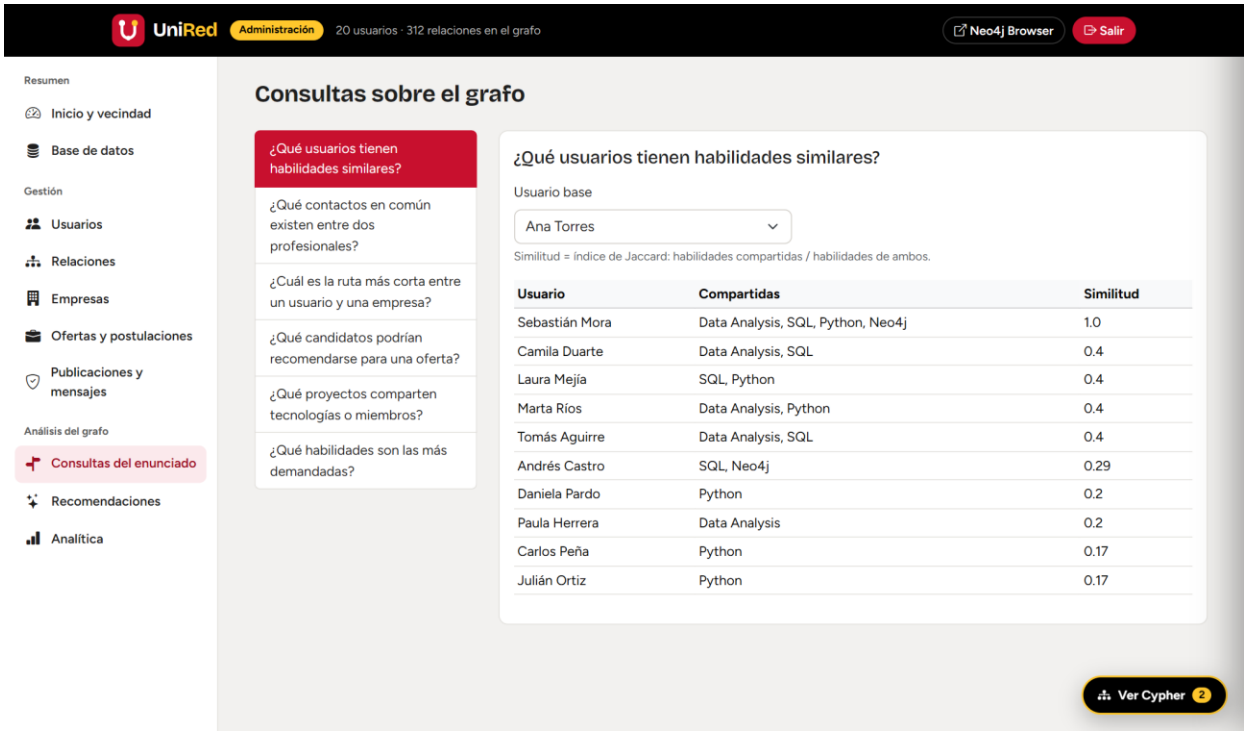


Figura 2. *Consulta C1 en el panel: usuarios con habilidades similares a Ana Torres.*

5.3 ¿Qué contactos en común existen entre dos profesionales?

```
MATCH (a:Usuario {id:'u1'})-[:CONECTA_CON]-(comun:Usuario)-[:CONECTA_CON]-(b:Usuario {id:'u8'})
RETURN comun.nombre AS contacto_en_comun, comun.titular AS titular;
```

Un patrón de dos saltos con un nodo intermedio. En SQL requeriría un auto-JOIN de la tabla de conexiones.

5.4 ¿Cuál es la ruta más corta entre un usuario y una empresa?

```
MATCH (u:Usuario {id:'u6'}), (e:Empresa {nombre:'CloudNine'}),
      p = shortestPath((u)-
[:CONECTA_CON|TRABAJA_EN|PARTICIPA_EN|POSTULA_A|PUBLICA|REQUIERE|TIENE_HABILIDAD|USA_TECNOLOGIA*.
.8]-(e))
RETURN p, length(p) AS saltos,
       [n IN nodes(p) | coalesce(n.nombre, n.titulo)] AS camino;
```

`shortestPath` busca el camino mínimo (hasta 8 saltos) sobre las relaciones del modelo profesional: puede pasar por contactos, empleos, proyectos, ofertas o habilidades, pero no por invitaciones pendientes, publicaciones ni mensajes. La variante C3b limita el recorrido a contactos y empleos.

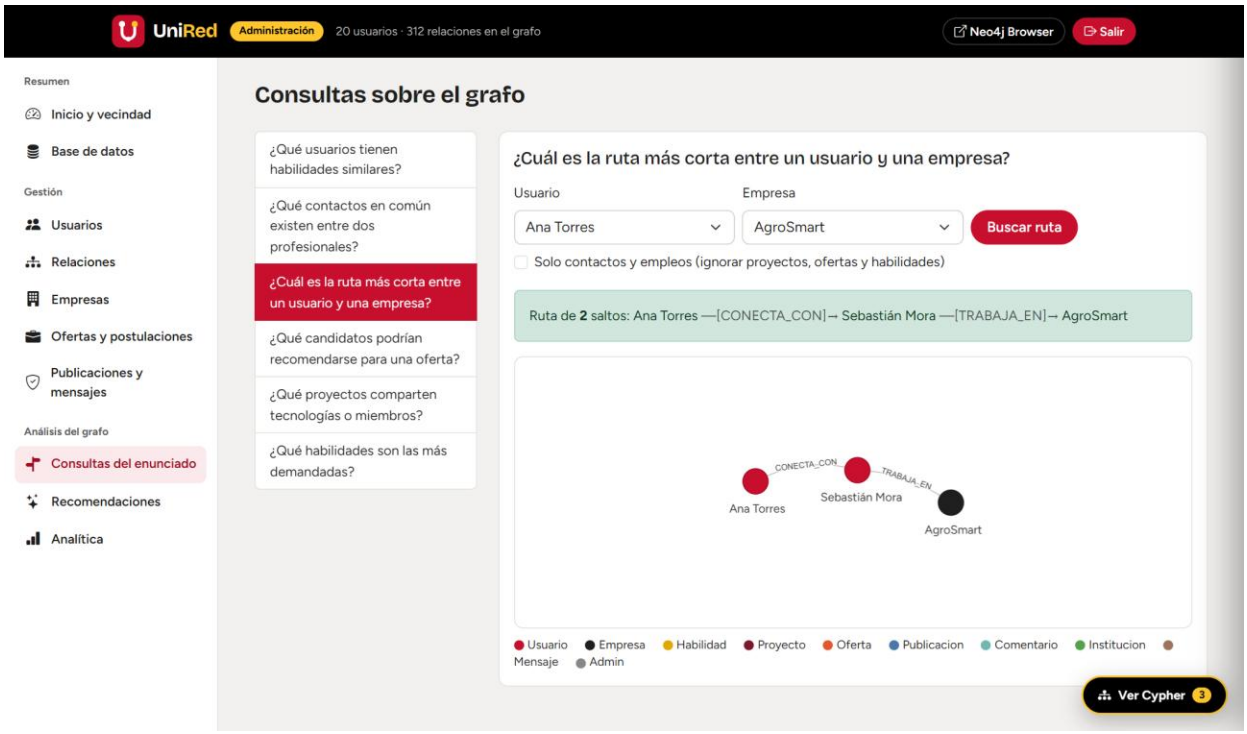


Figura 3. *Ruta más corta dibujada en el panel: Ana Torres llega a AgroSmart en 2 saltos.*

5.5 ¿Qué candidatos podrían recomendarse para una oferta?

```
MATCH (o:Oferta {id:'o3'})-[:REQUIERE]->(req:Habilidad)
WITH o, collect(req) AS requeridas
MATCH (o)<-[:PUBLICA]-(emp:Empresa)
MATCH (u:Usuario)
WHERE NOT (u)-[:TRABAJA_EN]->(emp) AND NOT (u)-[:POSTULA_A]->(o)
WITH o, emp, requeridas, u,
     [h IN requeridas WHERE (u)-[:TIENE_HABILIDAD]->(h) | h.nombre] AS cumple
WHERE size(cumple) > 0
OPTIONAL MATCH (u)-[:CONECTA_CON]-(colega:Usuario)-[:TRABAJA_EN]->(emp)
WITH u, requeridas, cumple, count(DISTINCT colega) AS conexiones_empresa
RETURN u.nombre AS candidato, cumple AS habilidades_cumplidas, conexiones_empresa,
       round(100 * (0.7 * size(cumple) / size(requeridas)
                    + 0.3 * toFloat(CASE WHEN conexiones_empresa > 3 THEN 3 ELSE
conexiones_empresa END) / 3)) AS puntaje
ORDER BY puntaje DESC LIMIT 10;
```

Combina contenido y estructura social: 70 % de cobertura de las habilidades requeridas y 30 % de contactos que el candidato tiene dentro de la empresa (tope de 3). Se excluyen empleados actuales y quienes ya se postularon.

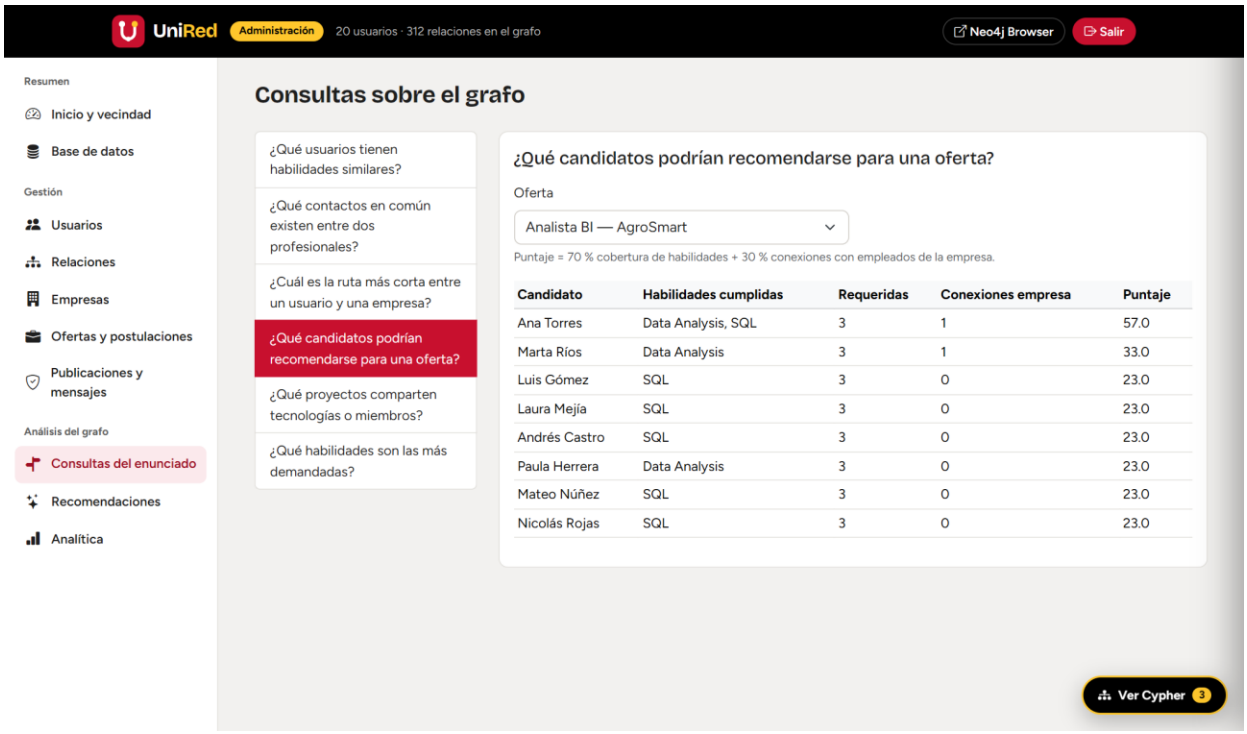


Figura 4. *Candidatos recomendados para una oferta, con su puntaje.*

5.6 ¿Qué proyectos comparten tecnologías o miembros?

```
MATCH (p1:Proyecto)-[:USA_TECNOLOGIA]->(h:Habilidad)<-[:USA_TECNOLOGIA]-(p2:Proyecto)
WHERE p1.id < p2.id
OPTIONAL MATCH (p1)<-[:PARTICIPA_EN]-(m:Usuario)-[:PARTICIPA_EN]->(p2)
RETURN p1.nombre AS proyecto_1, p2.nombre AS proyecto_2,
       collect(DISTINCT h.nombre) AS tecnologias_comunes,
       collect(DISTINCT m.nombre) AS miembros_comunes
ORDER BY size(collect(DISTINCT h.nombre)) DESC LIMIT 10;
```

5.7 ¿Qué habilidades son las más demandadas?

```
MATCH (o:Oferta {estado:'ABIERTA'})-[:REQUIERE]->(h:Habilidad)
RETURN h.nombre AS habilidad, count(o) AS ofertas
ORDER BY ofertas DESC, habilidad LIMIT 10;
```

Agregación sobre las ofertas abiertas. La variante C6b contrasta la demanda con la cantidad de profesionales que tienen cada habilidad (brecha de talento).

5.8 Recomendaciones

Contactos (amigos de amigos):

```
MATCH (yo:Usuario {id:'u1'})-[:CONECTA_CON]-(:Usuario)-[:CONECTA_CON]-(sug:Usuario)
WHERE sug <> yo AND NOT (yo)-[:CONECTA_CON]-(sug)
RETURN sug.nombre AS sugerido, sug.titular AS titular, count(*) AS contactos_comunes
ORDER BY contactos_comunes DESC LIMIT 5;
```

Empleos (ofertas abiertas con más habilidades del usuario) y **colaboradores** (personas en proyectos con tecnologías afines) completan las consultas D2 y D3 del script.

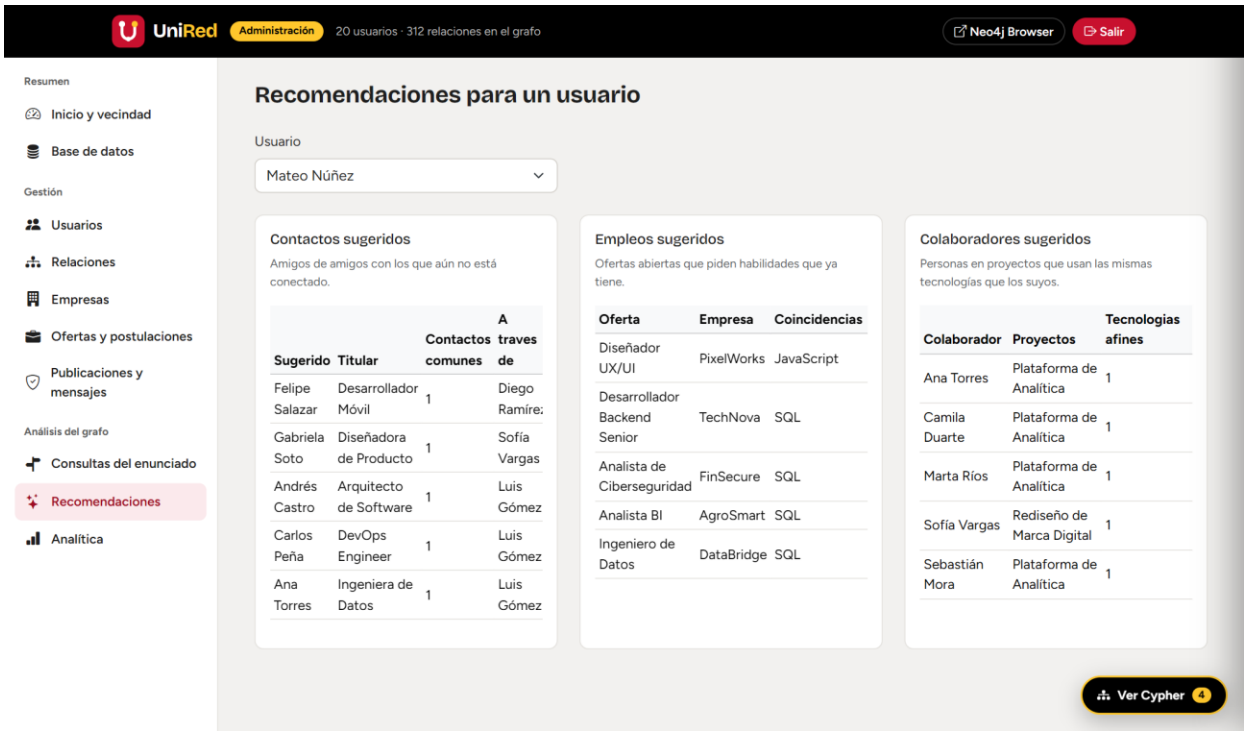


Figura 5. *Recomendaciones de contactos, empleos y colaboradores para un usuario.*

5.9 Analítica

```
MATCH (u:Usuario)
OPTIONAL MATCH (u)-[:CONECTA_CON]-(c:Usuario)
RETURN u.nombre AS usuario, count(DISTINCT c) AS conexiones
ORDER BY conexiones DESC LIMIT 10;
```

En la aplicación se complementa con **PageRank** y detección de **comunidades** (modularidad voraz de networkx) sobre las aristas `CONECTA_CON` leídas desde Neo4j. Con el plugin Graph Data Science se obtienen los mismos resultados con `gds.pageRank` y `gds.louvain`; el script lo incluye comentado.

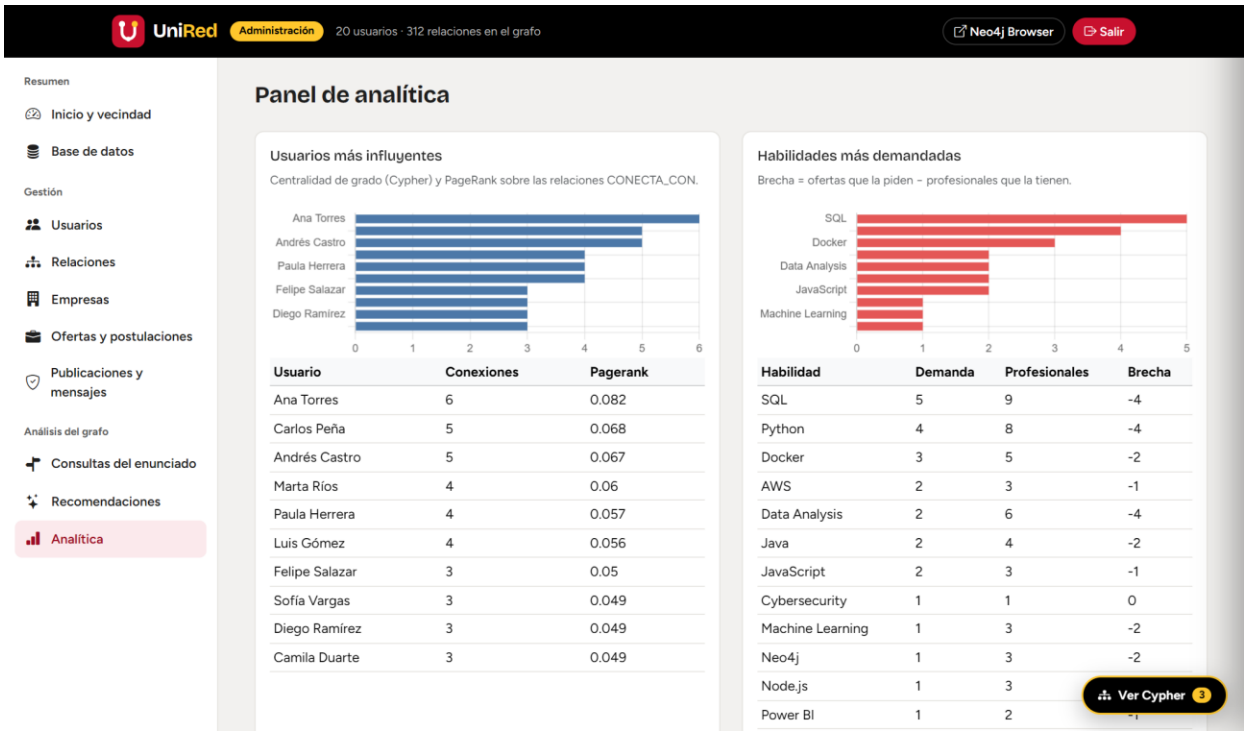


Figura 6. *Panel de analítica: usuarios más influyentes y habilidades más demandadas.*

5.10 Consultas de la capa social

Personas que quizá conozcas. Un puntaje que suma contactos en común (×3), instituciones compartidas (×2) y habilidades compartidas (×1), todo calculado recorriendo el grafo:

```
MATCH (yo:Usuario {id:$yo}), (s:Usuario)
WHERE s <> yo AND NOT (yo)-[:CONECTA_CON|SOLICITA_CONEXION]-(s)
WITH yo, s,
     COUNT { (yo)-[:CONECTA_CON]-(:Usuario)-[:CONECTA_CON]-(s) } AS comunes,
     COLLECT { MATCH (yo)-[:ESTUDIO_EN]->(i:Institucion)<-[:ESTUDIO_EN]-(s) RETURN DISTINCT
i.nombre } AS instituciones,
     COUNT { (yo)-[:TIENE_HABILIDAD]->(:Habilidad)<-[:TIENE_HABILIDAD]-(s) } AS habilidades
WITH s, comunes, instituciones, habilidades, comunes * 3 + size(instituciones) * 2 + habilidades
AS puntaje
WHERE puntaje > 0
RETURN s.nombre, comunes, instituciones, habilidades, puntaje ORDER BY puntaje DESC LIMIT 12;
```

Grado de conexión y «Cómo llegas a…». El perfil de otra persona muestra si es contacto de 1.º, 2.º o 3.er grado y el camino real de contactos:

```
MATCH (yo:Usuario {id:$yo}), (u:Usuario {id:$u})
MATCH p = shortestPath((yo)-[:CONECTA_CON*..6]-(u))
RETURN length(p) AS grado, [n IN nodes(p) | n.nombre] AS camino;
```

Aceptar una invitación convierte la relación pendiente en una conexión real:

```
MATCH (o:Usuario {id:$o})-[s:SOLICITA_CONEXION]->(yo:Usuario {id:$yo})
DELETE s
WITH o, yo WHERE $aceptar AND NOT (o)-[:CONECTA_CON]-(yo)
CREATE (o)-[:CONECTA_CON {desde: date()}]->(yo);
```

Mensajes que vencen. Solo se puede escribir a un contacto (el `MATCH` exige `CONECTA_CON`) y la limpieza borra lo que tenga más de 24 horas:

```
MATCH (yo:Usuario {id:$yo})-[:CONECTA_CON]-(o:Usuario {id:$o})
CREATE (yo)-[:ENVIO]->(m:Mensaje {id:$id, texto:$texto, fecha: datetime(), leido:false})-[:PARA]->(o);

MATCH (m:Mensaje) WHERE m.fecha < datetime() - duration({hours: 24})
DETACH DELETE m;
```

6. Aplicación UniRed

La aplicación web está construida con Flask y se conecta a Neo4j mediante el controlador oficial. Los datos de conexión (URI, usuario y contraseña) se leen de un archivo `.env`, de modo que el mismo código funciona con Neo4j local, Neo4j Desktop o AuraDB. Todas las consultas Cypher están en los módulos `db.py` (enunciado) y `red.py` (capa social), y siempre reciben parámetros, nunca texto concatenado, lo que evita la inyección de Cypher.

6.1 Arquitectura

Archivo	Responsabilidad
app.py	Arranque de Flask, inicio de sesión y registro, protección CSRF, registro de consultas para «Ver Cypher», limpieza de mensajes cada 10 min
db.py	Conexión a Neo4j y todas las consultas del enunciado; consola de solo lectura
red.py	Consultas de la capa social (feed, reacciones, perfil, red, empleos, mensajes, notificaciones)
rutas_admin.py / rutas_red.py	Pantallas del panel de administración y de UniRed
cargar_datos.py	Ejecuta los scripts 01, 02 y 04, asigna claves de demostración y crea el administrador
servidor.py	Servidor de producción (waitress) que se publica con el túnel

6.2 Inicio de sesión y registro

Cualquier persona puede crear una cuenta con nombre, correo, contraseña (mínimo 8 caracteres), titular, ciudad y dónde estudia; eso crea el nodo `Usuario` y su relación `ESTUDIO_EN`. Las contraseñas se guardan como hash, nunca en texto plano. El mismo formulario sirve a los administradores, que llegan directamente al panel. Al entrar se muestra una pantalla de carga con el logo animado.

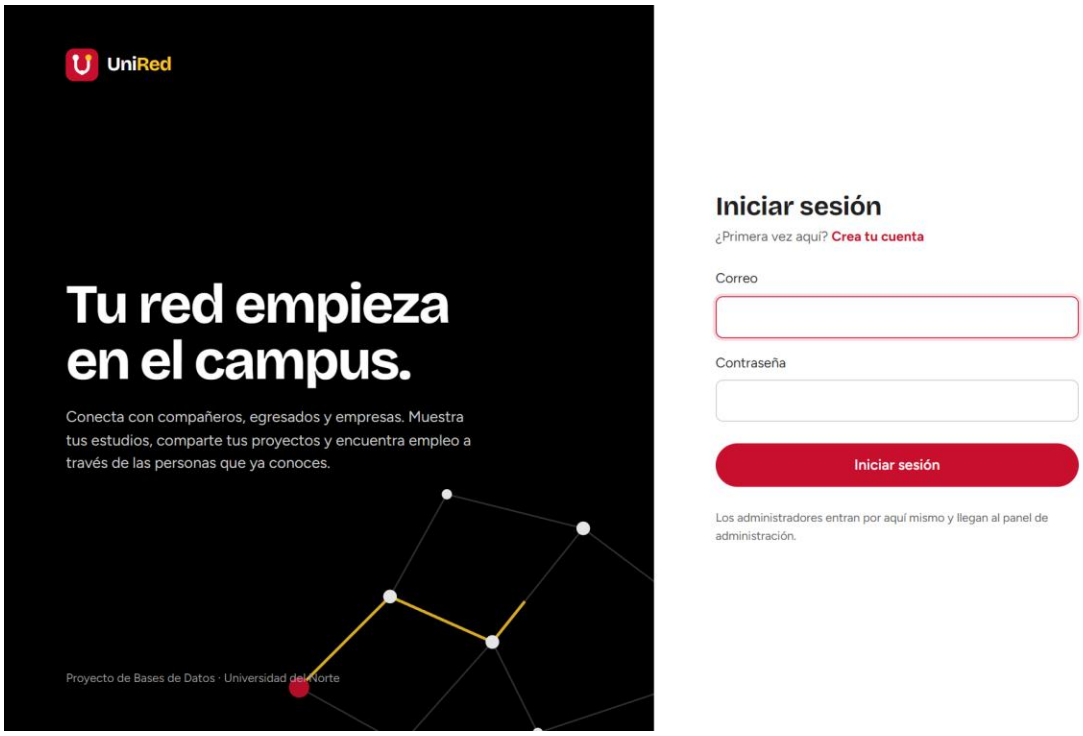


Figura 7. *Pantalla de inicio de sesión de UniRed.*

6.3 Feed, publicaciones y reacciones

El feed muestra las publicaciones (todas o solo las de mi red) con texto, foto o video. Cada publicación admite seis reacciones (recomendar, celebrar, apoyar, me encanta, interesante y divertido) que se guardan como la propiedad `tipo` de la relación `REACCIONO`, y comentarios (`ESCRIBIO` → `SOBRE`). A los lados aparecen la tarjeta del usuario, «Personas que quizá conozcas» y las habilidades más pedidas en las ofertas.

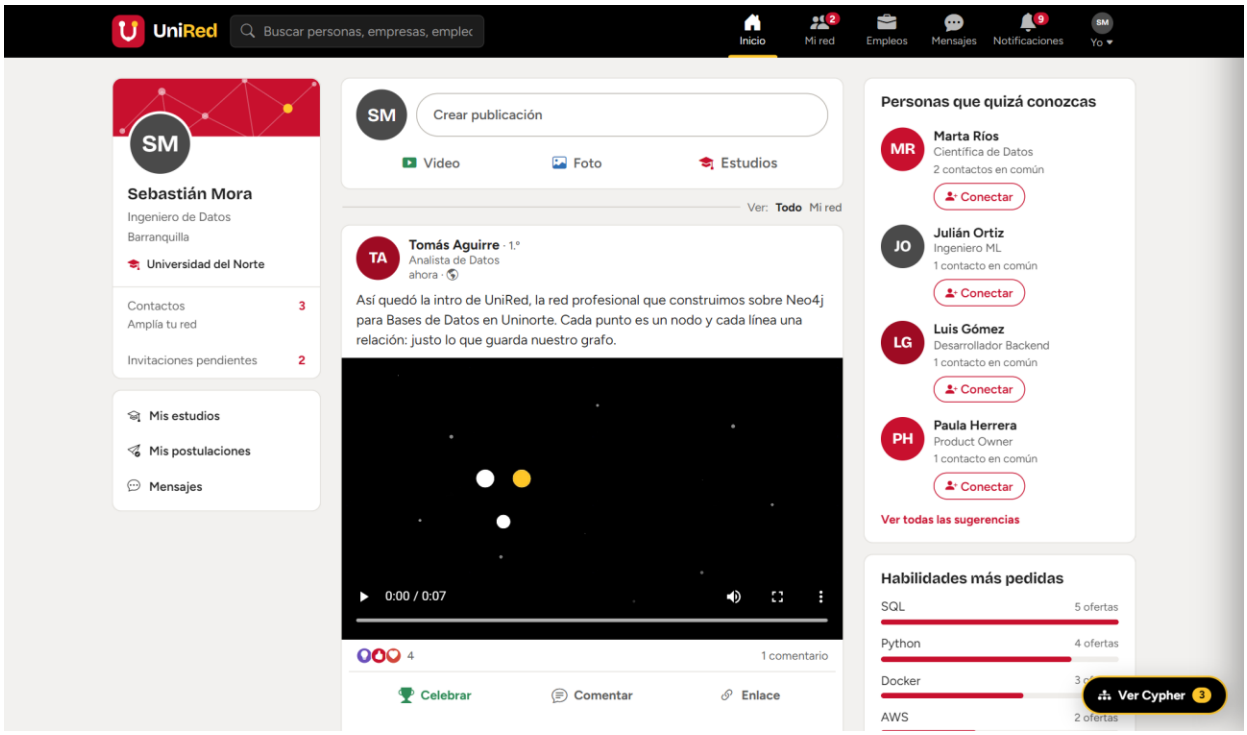


Figura 8. *Feed de UniRed con publicaciones, sugerencias de contactos y habilidades más pedidas.*



Figura 9. *Paleta de reacciones; la elegida se guarda en la relación REACCIONO.*

6.4 Perfil, estudios y «Cómo llegas a...»

El perfil reúne experiencia (`TRABAJA_EN`), educación (`ESTUDIO_EN`), habilidades con nivel (`TIENE_HABILIDAD`), proyectos y publicaciones. Si es el perfil de otra persona, muestra el grado de conexión y el camino más corto de contactos para llegar a ella, además de los contactos en común.

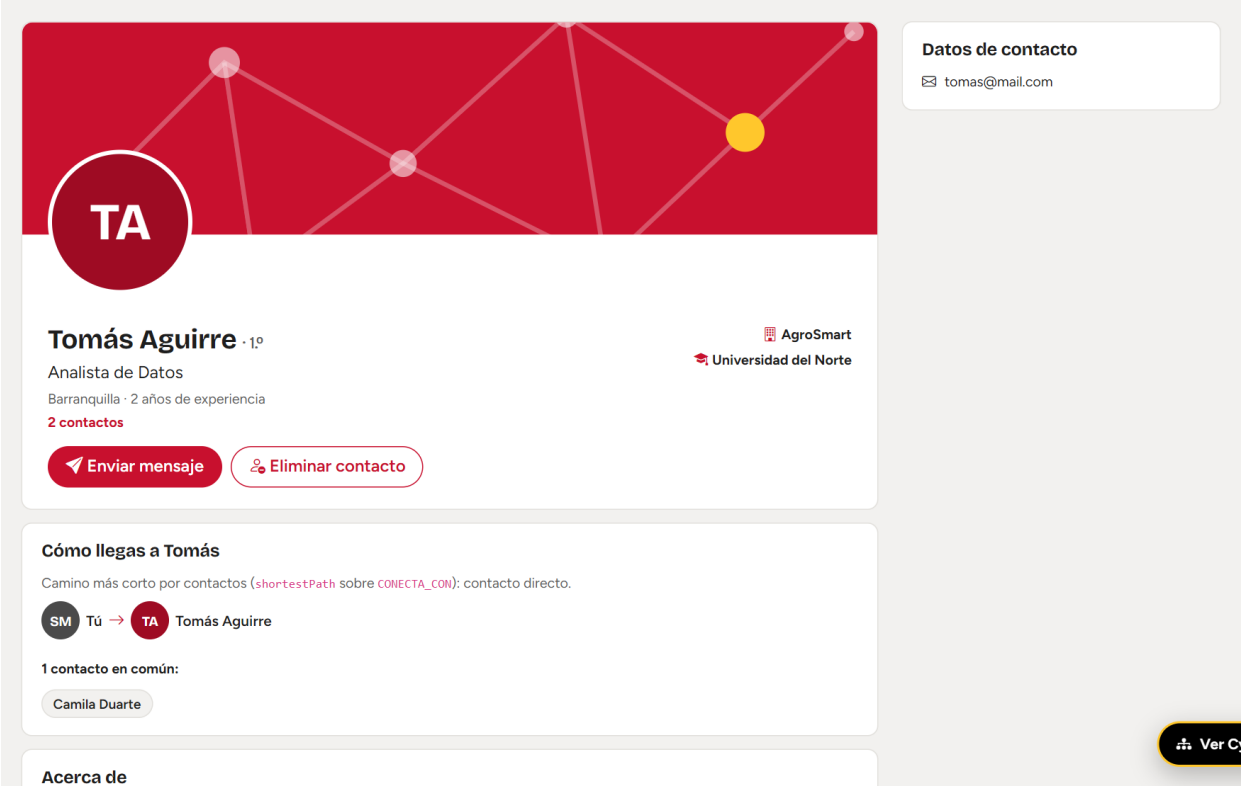


Figura 10. *Perfil de Tomás Aguirre visto por Sebastián: grado de conexión, camino y contactos en común.*

6.5 Mi red, empleos y mensajes

Mi red muestra las invitaciones recibidas y enviadas, los contactos y las sugerencias con el motivo (contactos en común, misma institución, habilidades). **Empleos** ordena las ofertas abiertas por porcentaje de ajuste con las habilidades del usuario, resalta las que ya tiene y cuenta cuántos contactos trabajan en la empresa; quien trabaja en una empresa puede publicar ofertas y cambiar el estado de los postulantes. **Mensajes** permite conversar en vivo solo con contactos; los mensajes se borran a las 24 horas.

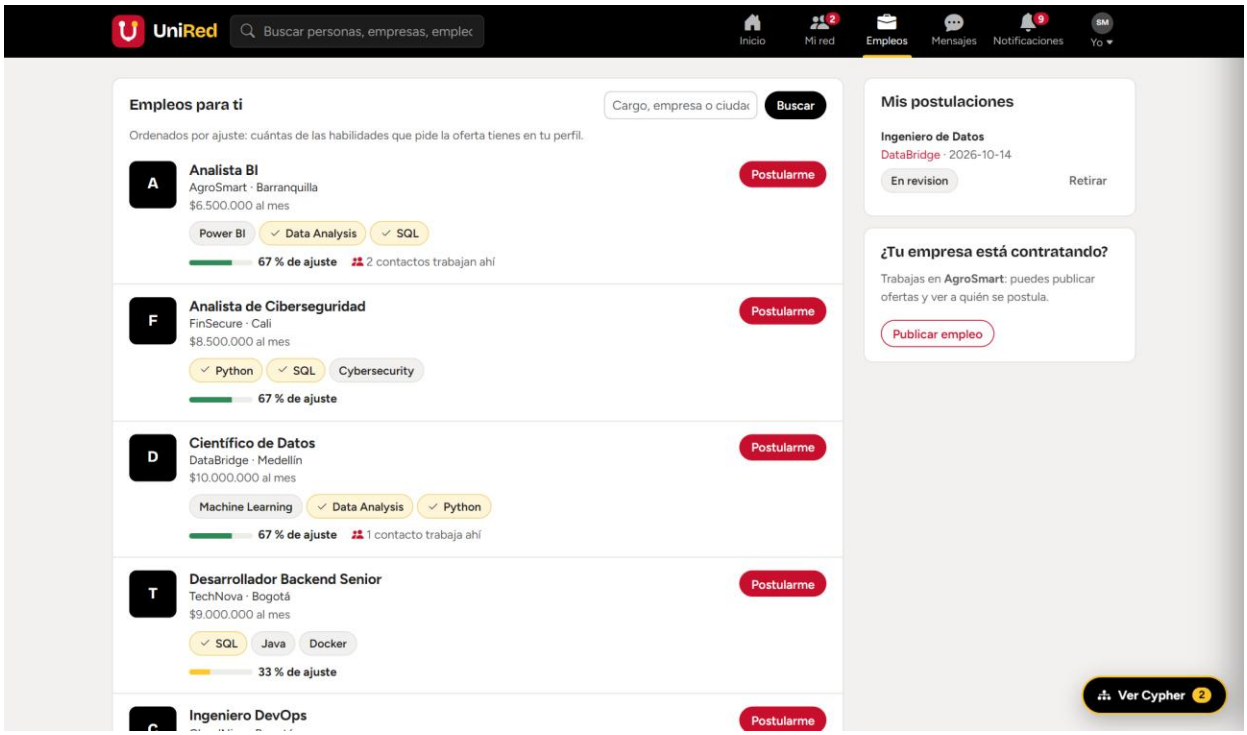


Figura 11. *Empleos para ti: ajuste por habilidades y contactos que trabajan en la empresa.*

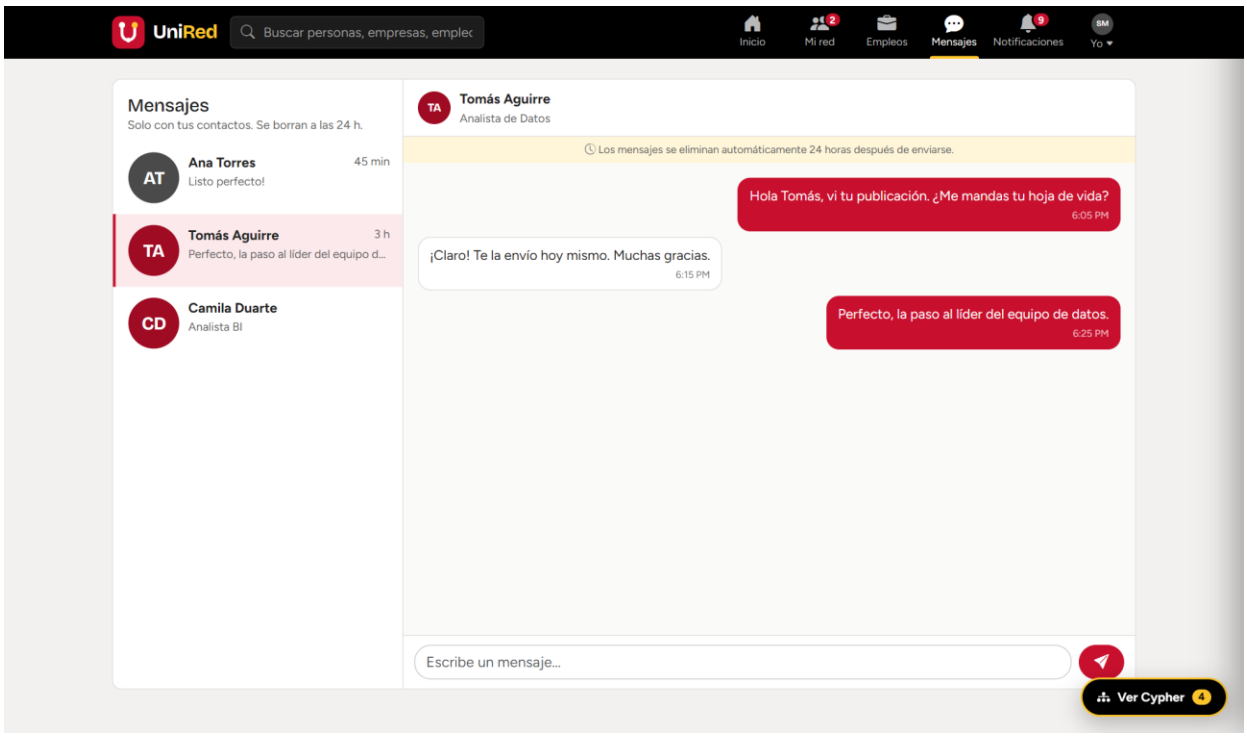


Figura 12. *Mensajería entre contactos con aviso de borrado a las 24 horas.*

6.6 Panel de administración

Sección	Funcionalidad
Inicio y vecindad	Contadores del grafo y visualización interactiva de la vecindad de 2 saltos de un usuario
Base de datos	Nodos por etiqueta, relaciones por tipo con su origen y destino, consola Cypher de solo lectura, grafo completo y restricciones
Usuarios	Listar, ver perfil, crear, editar y eliminar usuarios
Relaciones	Conectar usuarios; asignar habilidades; vincular con empresa y proyecto; crear habilidades y proyectos
Empresas, ofertas y postulaciones	Registrar empresas, publicar ofertas con habilidades requeridas, postular candidatos
Publicaciones y mensajes	Moderar publicaciones y comentarios; contar mensajes (no se leen) y forzar la limpieza de 24 h
Consultas del enunciado	Las seis preguntas; la ruta más corta se dibuja
Recomendaciones y analítica	Contactos, empleos y colaboradores; influyentes, habilidades demandadas y brecha, comunidades

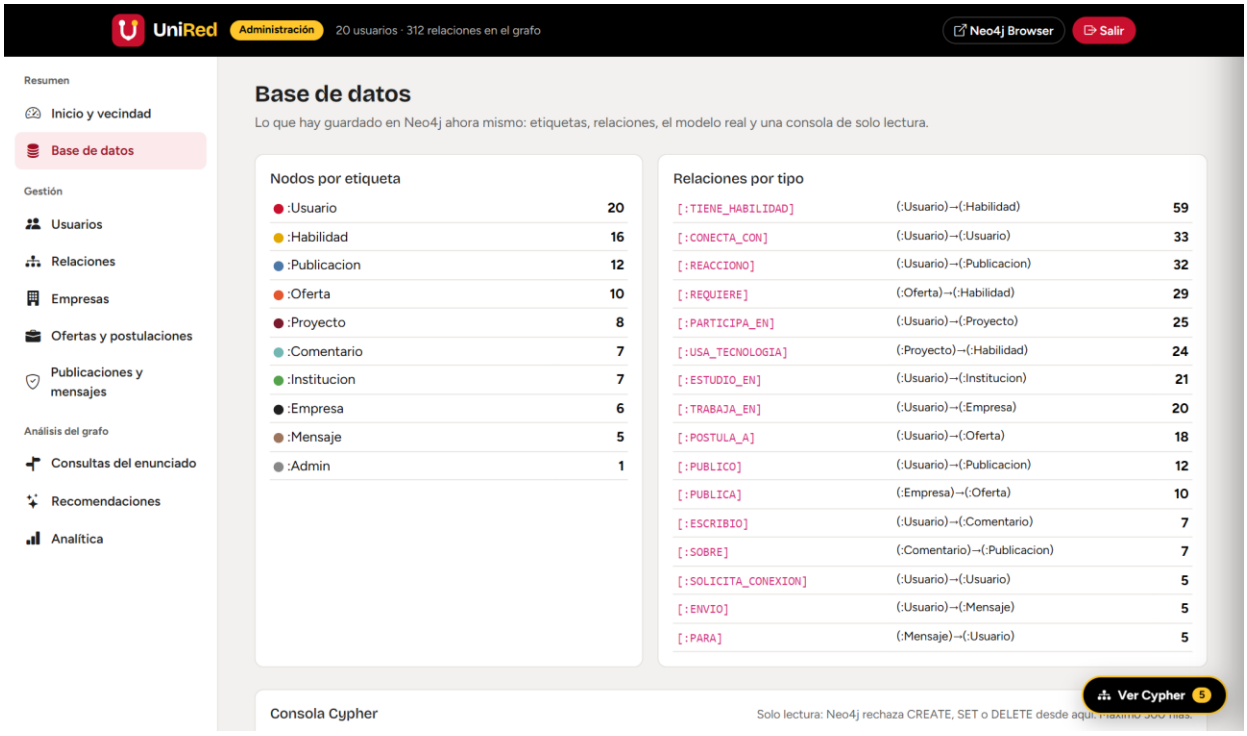


Figura 13. *Panel › Base de datos: conteos por etiqueta y tipo de relación, consola y grafo completo.*

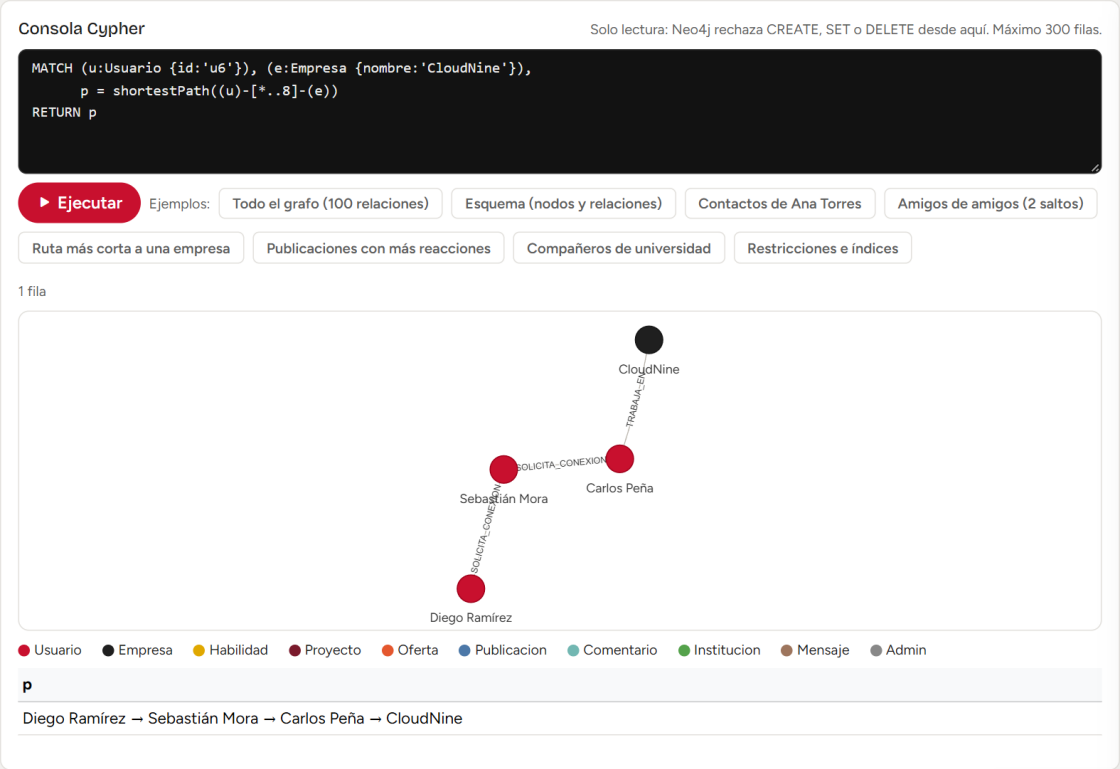


Figura 14. *Consola Cypher de solo lectura: el resultado se muestra como tabla y como grafo.*

6.7 «Ver Cypher»

Un botón flotante en todas las pantallas abre un panel con las consultas Cypher que se enviaron a Neo4j para construir la página y las de la última acción (por ejemplo, publicar o reaccionar), con sus parámetros. Las contraseñas aparecen ocultas. Sirve para explicar en la demostración qué hace la base de datos detrás de cada clic.

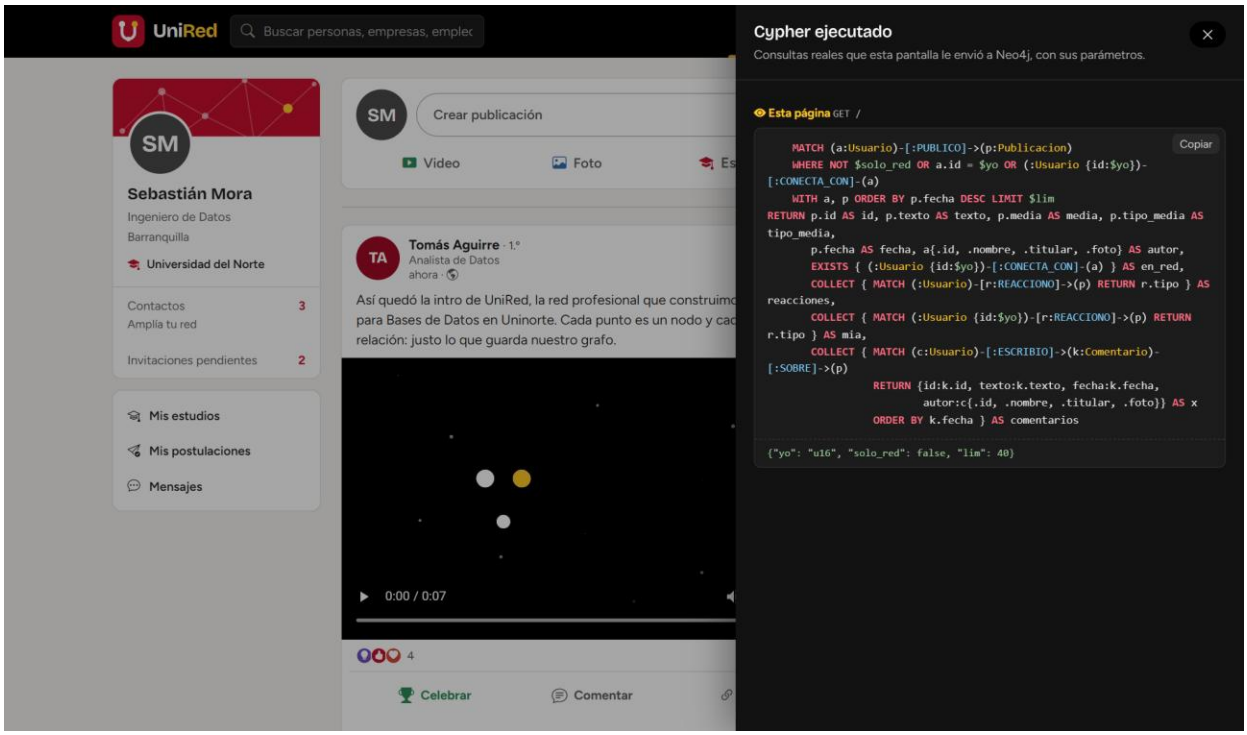


Figura 15. *Panel «Ver Cypher» con las consultas reales del feed.*

6.8 Seguridad

- Contraseñas con hash (PBKDF2) y bloqueo temporal tras 8 intentos fallidos de inicio de sesión.
- Token CSRF en todos los formularios; la sesión expira a los 7 días.
- Consultas siempre parametrizadas; la consola del panel abre la sesión en modo **solo lectura**, así que Neo4j rechaza `CREATE`, `SET` o `DELETE` desde allí.
- Reglas de negocio en el propio patrón Cypher: solo se escribe a contactos, solo el autor borra su publicación y solo quien trabaja en la empresa ve a sus postulantes.
- Archivos subidos con nombre aleatorio, extensiones permitidas (imágenes y videos) y tamaño máximo de 80 MB.

6.9 Versión móvil

La interfaz se adapta al celular: la barra de secciones pasa a la parte inferior y las columnas se apilan. En pantallas táctiles, mantener presionado «Recomendar» abre la paleta de reacciones.



Figura 16. *UniRed en un celular.*

6.10 Puesta en marcha y publicación

No se usan contenedores. En el PC del equipo, Neo4j Community 5.26 corre en modo portable desde `D:\neo4j_local` con su propio Java 21. Para encender todo:

1. Doble clic en `iniciar.bat` (carpeta del proyecto). Enciende Neo4j, la aplicación (waitress en el puerto 8000) y un túnel de Cloudflare.
2. El script muestra el **enlace público https** (también queda copiado y guardado en `LINK_PUBLICO.txt`). Con él se entra desde cualquier computador o celular. El enlace cambia cada vez que se enciende.
3. `detener.bat` apaga todo. Para reiniciar los datos de ejemplo: `python cargar_datos.py --reset`.

Instalación desde cero en otro equipo: `pip install -r requirements.txt`, copiar `.env.example` como `.env` con la URI, el usuario y la contraseña de Neo4j (y `ADMIN_PASSWORD`), ejecutar `python cargar_datos.py` y luego `python app.py` (desarrollo) o `python servidor.py` (producción).

6.11 Ver la base de datos en Neo4j Browser

Neo4j Browser cumple el papel que tiene MySQL Workbench en el modelo relacional. Con Neo4j encendido se abre `http://localhost:7474` y se conecta con la URL `neo4j://localhost:7687`, el usuario `neo4j` y la contraseña configurada. Las consultas se escriben en la barra superior y se ejecutan con Ctrl+Enter; cada resultado puede verse como **Graph**, **Table** o **Text**.

En Workbench (MySQL)	En Neo4j Browser
Ver las tablas	Ícono de base de datos: etiquetas, tipos de relación y propiedades (clic para consultarlas)
SELECT * FROM usuario	`MATCH (u:Usuario) RETURN u`
Diagrama EER	`CALL db.schema.visualization()`
Ver todo	`MATCH (a)-[r]->(b) RETURN a, r, b LIMIT 150`

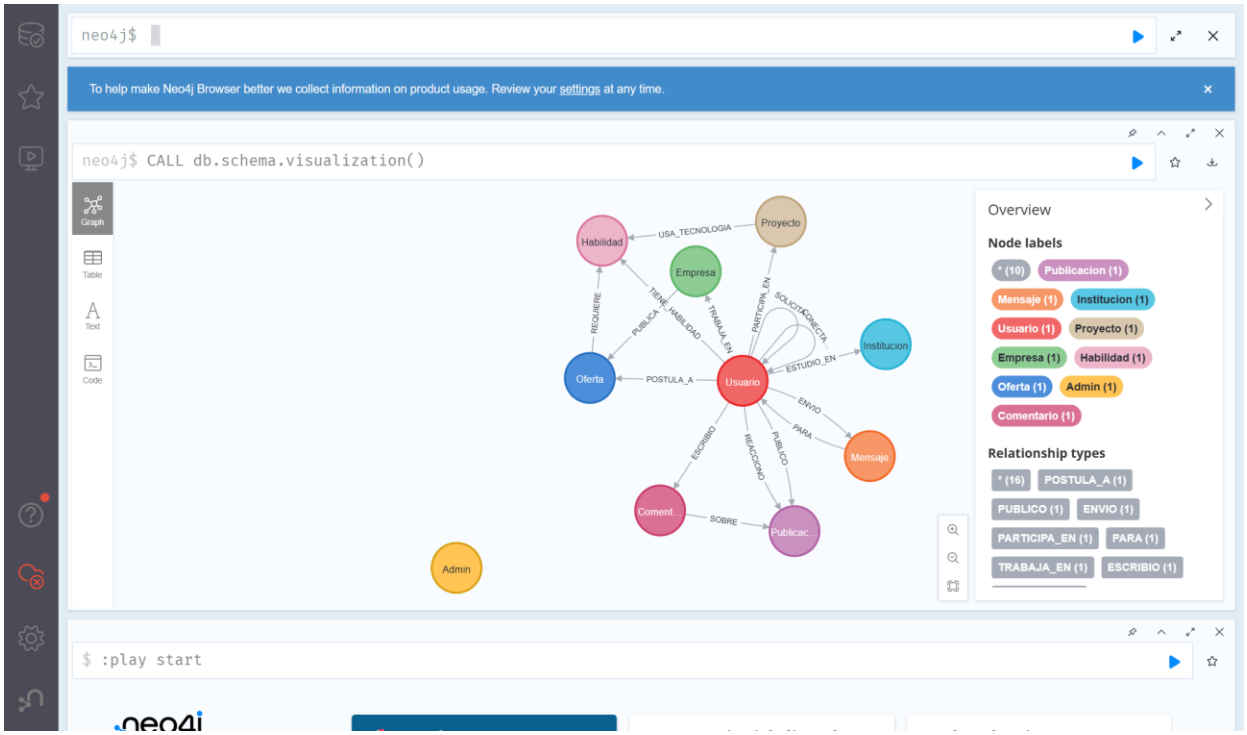


Figura 17. *CALL db.schema.visualization() en Neo4j Browser: el modelo real guardado en la base.*

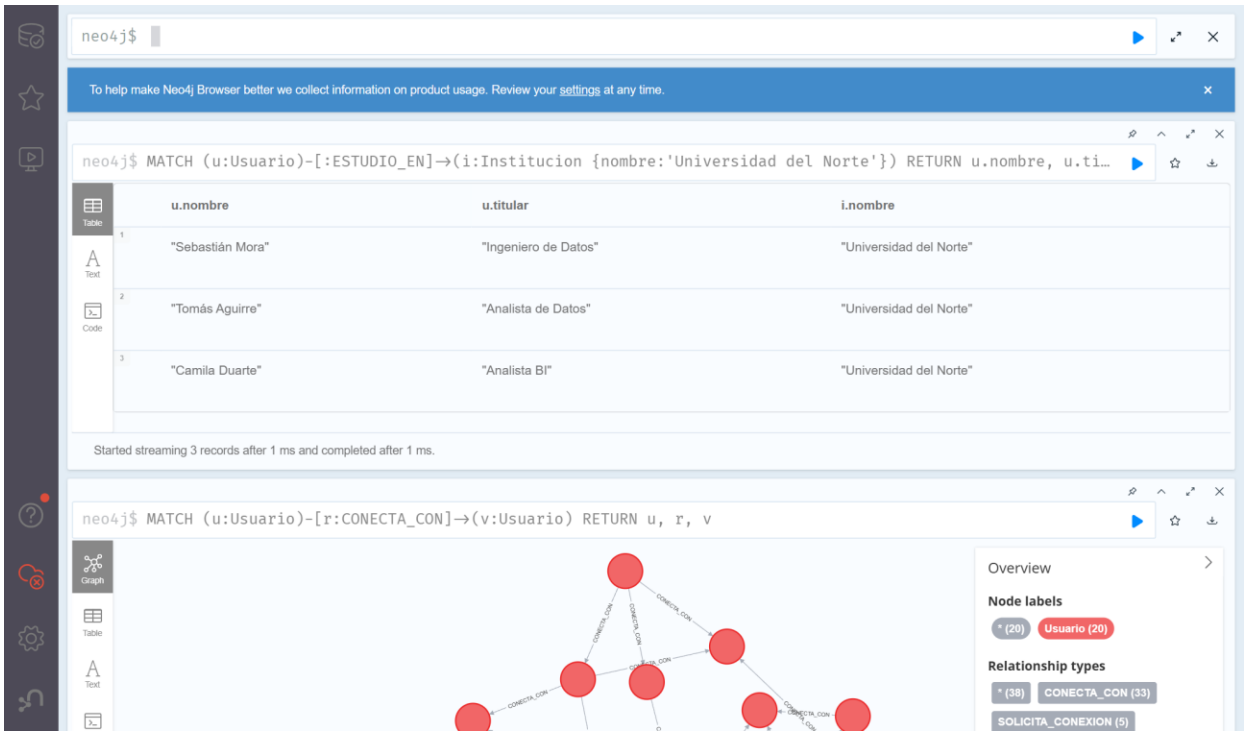


Figura 18. *Consulta en Neo4j Browser con resultado en tabla y la red de contactos como grafo.*

7. Componente de investigación: grafos frente a relacional

7.1 El problema de los JOIN

En un esquema relacional, la red de contactos se modela con una tabla de usuarios y una tabla intermedia `conexion(usuario_a, usuario_b)`. Obtener «los contactos de mis contactos» exige un auto-JOIN, y cada nivel adicional agrega otro:

```
-- Amigos de amigos de amigos (3 niveles) en SQL
SELECT DISTINCT u3.nombre
FROM conexion c1
JOIN conexion c2 ON c2.usuario_a = c1.usuario_b
JOIN conexion c3 ON c3.usuario_a = c2.usuario_b
JOIN usuario  u3 ON u3.id = c3.usuario_b
WHERE c1.usuario_a = 1;

// Mismo problema en Cypher: la profundidad es un parámetro
MATCH (:Usuario {id:'u1'})-[:CONECTA_CON*3]-(x:Usuario)
RETURN DISTINCT x.nombre;
```

El número de filas intermedias en SQL crece de forma combinatoria con la profundidad y la consulta debe reescribirse para cada nivel. Para el camino más corto sin límite conocido se necesita una consulta recursiva (`WITH RECURSIVE`), más difícil de escribir, de optimizar y de leer. En el grafo, `shortestPath` es una operación nativa. En UniRed esto se nota en «Cómo llegas a…», que se calcula en cada visita a un perfil.

7.2 Cuándo el modelo de grafos supera al relacional

Escenario	Por qué el grafo gana
Recorridos de profundidad variable (amigos de amigos, rutas, dependencias)	Recorrer una relación es un salto de puntero; el costo depende del subgrafo visitado, no del tamaño total
Relaciones muchos-a-muchos y heterogéneas (usuario–habilidad–oferta–empresa)	Cada tipo de relación es de primera clase, sin tablas intermedias ni JOIN encadenados
Recomendación y similitud basadas en estructura	Los patrones (triángulos, vecinos compartidos) se expresan directamente
Análisis de redes: centralidad, comunidades, caminos	Existen algoritmos y librerías (GDS) que operan sobre la estructura nativa
Esquema cambiante	Agregar un tipo de relación o de propiedad no exige migraciones (ALTER TABLE). La capa social de UniRed se añadió sin tocar los nodos del enunciado
Consultas «exploratorias» con el patrón visual	Cypher describe el patrón que se dibuja en la pizarra

7.3 Cuándo no conviene (o el relacional es mejor)

Escenario	Por qué el relacional gana
Agregaciones masivas sobre toda la tabla (sumas, promedios, reportes contables)	Los motores relacionales y columnares están optimizados para escaneos y GROUP BY a gran escala
Datos tabulares sin relaciones relevantes (inventario, nómina)	El grafo no aporta; añade complejidad
Transacciones con integridad muy estricta entre muchas entidades	El modelo relacional ofrece claves foráneas, CHECK y normalización con garantías declarativas maduras (Neo4j Community solo ofrece unicidad)
Escalado horizontal (sharding) de grandes volúmenes	Particionar un grafo es difícil porque las consultas cruzan particiones
Ecosistema y habilidades del equipo	SQL es un estándar con mayor soporte de herramientas, BI y personal
Consultas que tocan casi todo el grafo	Si cada consulta recorre la mayoría de los nodos, la ventaja de la adyacencia se pierde

7.4 Aplicado a este proyecto

En la red profesional las preguntas centrales (contactos en común, rutas hacia una empresa, candidatos según competencias y conexiones, proyectos relacionados, personas que quizá conozcas) son recorridos sobre relaciones múltiples, justo donde el grafo es natural. En cambio, un reporte como «salario promedio por ciudad y mes» sería una agregación tabular en la que un motor relacional rendiría igual o mejor. Algo parecido ocurre con los mensajes: son registros temporales que se consultan por fecha y se borran en lote, y un motor relacional los manejaría con la misma comodidad. En un sistema real la

elección puede ser híbrida (persistencia políglota): relacional para datos transaccionales y contables, y grafo para la capa de relaciones y recomendación.

7.5 Comparación breve

Criterio	Grafos (Neo4j)	Relacional (SQL)
Modelo	Nodos y relaciones con propiedades	Tablas, filas y claves foráneas
Relaciones	Primera clase, sin JOIN	Se reconstruyen con JOIN
Consulta de caminos	Nativa (shortestPath, [*n..m])	WITH RECURSIVE, costosa
Esquema	Flexible	Rígido (migraciones)
Agregaciones masivas	Aceptables	Muy optimizadas
Integridad	Restricciones básicas	Muy completa y declarativa
Escalado horizontal	Complejo	Variable, madura en muchos motores

Nota: la comparación es cualitativa. No se realizó un benchmark de tiempos; puede medirse cargando un grafo sintético grande y comparando una consulta de 3 a 5 niveles en Cypher frente a su equivalente en SQL.

8. Guion de la demostración (15 min)

Minutos	Segmento	Qué mostrar
0–2	Contexto y modelo	Logo animado; figura del modelo; por qué un grafo
2–4	Base de datos	Neo4j Browser: `CALL db.schema.visualization()` y el grafo; Panel › Base de datos
4–8	UniRed como usuario	Sebastián Mora: feed, reaccionar, comentar, publicar, aceptar invitación, perfil con «Cómo llegas a…», empleos, chat en vivo con Tomás
8–12	Panel de administración	CRUD de un usuario; las seis consultas (ruta más corta, candidatos); recomendaciones; analítica. «Ver Cypher» en cada paso
12–15	Grafos vs. relacional	Amigos de amigos en SQL frente a Cypher; cuándo no usar grafos; conclusiones

Recomendación: encender con `iniciar.bat` antes de la presentación, ensayar con un reinicio limpio de los datos y tener Neo4j Browser abierto en otra pestaña. Las cuentas de la demo están en un documento aparte de uso interno del equipo.

9. Conclusiones

- El modelo de grafo de propiedades permitió representar de forma directa las cinco entidades y las relaciones del enunciado, y las consultas de caminos, similitud y recomendación se expresan con patrones breves.
- La capa social (publicaciones, reacciones, comentarios, invitaciones, estudios y mensajes) se agregó sin modificar el núcleo del modelo, lo que muestra la flexibilidad de esquema del grafo.
- Cypher reduce la complejidad de las consultas de profundidad variable frente a los JOIN encadenados o recursivos de SQL; funciones como `shortestPath` y las subconsultas `COUNT { }` resuelven en una consulta lo que en SQL requiere varias.
- Modelar bien el significado de cada relación importa: una invitación pendiente no es una conexión, y por eso los recorridos se restringen a los tipos de relación que tienen sentido para cada pregunta.
- El grafo no sustituye al modelo relacional en agregaciones masivas ni en integridad transaccional compleja; una arquitectura híbrida suele ser la mejor opción.
- Pendientes recomendados: pruebas de rendimiento con un grafo mayor, notificaciones en tiempo real y uso de GDS para analítica a mayor escala.

Referencias

- Neo4j. Cypher Manual. https://neo4j.com/docs/cypher-manual/current/
- Neo4j. Operations Manual: constraints and indexes. https://neo4j.com/docs/
- Robinson, I., Webber, J., Eifrem, E. (2015). Graph Databases (2.ª ed.). O'Reilly Media.
- Hagberg, A., Schult, D., Swart, P. NetworkX documentation. https://networkx.org/
- Pallets. Flask documentation. https://flask.palletsprojects.com/

Anexo A. 01_schema.cypher

```
// ========================================================================
// 01_schema.cypher  -  Restricciones e índices (Neo4j 5.x)
// Ejecutar una sola vez antes de cargar datos.
// ========================================================================

CREATE CONSTRAINT usuario_id   IF NOT EXISTS FOR (u:Usuario)   REQUIRE u.id IS UNIQUE;
CREATE CONSTRAINT usuario_mail IF NOT EXISTS FOR (u:Usuario)   REQUIRE u.email IS UNIQUE;
CREATE CONSTRAINT empresa_id   IF NOT EXISTS FOR (e:Empresa)   REQUIRE e.id IS UNIQUE;
CREATE CONSTRAINT habilidad_nm IF NOT EXISTS FOR (h:Habilidad) REQUIRE h.nombre IS UNIQUE;
CREATE CONSTRAINT proyecto_id  IF NOT EXISTS FOR (p:Proyecto)  REQUIRE p.id IS UNIQUE;
CREATE CONSTRAINT oferta_id    IF NOT EXISTS FOR (o:Oferta)    REQUIRE o.id IS UNIQUE;

CREATE INDEX usuario_nombre IF NOT EXISTS FOR (u:Usuario) ON (u.nombre);
CREATE INDEX empresa_nombre IF NOT EXISTS FOR (e:Empresa) ON (e.nombre);
CREATE INDEX oferta_estado  IF NOT EXISTS FOR (o:Oferta)  ON (o.estado);

// ---------- Red social (UniRed) ----------
CREATE CONSTRAINT admin_email       IF NOT EXISTS FOR (a:Admin)       REQUIRE a.email IS UNIQUE;
CREATE CONSTRAINT publicacion_id    IF NOT EXISTS FOR (p:Publicacion) REQUIRE p.id IS UNIQUE;
CREATE CONSTRAINT comentario_id     IF NOT EXISTS FOR (k:Comentario)  REQUIRE k.id IS UNIQUE;
CREATE CONSTRAINT mensaje_id        IF NOT EXISTS FOR (m:Mensaje)     REQUIRE m.id IS UNIQUE;
CREATE CONSTRAINT institucion_nm    IF NOT EXISTS FOR (i:Institucion) REQUIRE i.nombre IS UNIQUE;

CREATE INDEX publicacion_fecha IF NOT EXISTS FOR (p:Publicacion) ON (p.fecha);
CREATE INDEX mensaje_fecha     IF NOT EXISTS FOR (m:Mensaje)     ON (m.fecha);
```

Anexo B. 02_seed.cypher

```
// ======================================================================
// 02_seed.cypher  -  Datos de ejemplo de la red profesional
// 20 usuarios, 6 empresas, 16 habilidades, 8 proyectos, 10 ofertas.
// Es idempotente (usa MERGE): se puede ejecutar varias veces.
// ======================================================================

// ---------- Habilidades ----------
UNWIND ['Python','Java','JavaScript','SQL','Neo4j','Docker','AWS','React',
        'Machine Learning','Data Analysis','Scrum','UX Design','Kotlin',
        'Cybersecurity','Node.js','Power BI'] AS nombre
MERGE (:Habilidad {nombre: nombre});

// ---------- Empresas ----------
UNWIND [
  {id:'e1', nombre:'TechNova',     sector:'Software',     ciudad:'Bogotá'},
  {id:'e2', nombre:'DataBridge',   sector:'Analítica',    ciudad:'Medellín'},
  {id:'e3', nombre:'CloudNine',    sector:'Nube',         ciudad:'Bogotá'},
  {id:'e4', nombre:'FinSecure',    sector:'Fintech',      ciudad:'Cali'},
  {id:'e5', nombre:'PixelWorks',   sector:'Diseño',       ciudad:'Medellín'},
  {id:'e6', nombre:'AgroSmart',    sector:'Agrotech',     ciudad:'Barranquilla'}
] AS r
MERGE (e:Empresa {id:r.id})
SET e.nombre=r.nombre, e.sector=r.sector, e.ciudad=r.ciudad;

// ---------- Usuarios ----------
UNWIND [
  {id:'u1',  nombre:'Ana Torres',      email:'ana@mail.com',      titular:'Ingeniera de Datos',
ciudad:'Bogotá',       anios:6},
  {id:'u2',  nombre:'Luis Gómez',      email:'luis@mail.com',     titular:'Desarrollador
Backend',     ciudad:'Bogotá',       anios:4},
  {id:'u3',  nombre:'Marta Ríos',      email:'marta@mail.com',    titular:'Científica de Datos',
ciudad:'Medellín',     anios:5},
  {id:'u4',  nombre:'Carlos Peña',     email:'carlos@mail.com',   titular:'DevOps Engineer',
ciudad:'Bogotá',       anios:7},
  {id:'u5',  nombre:'Sofía Vargas',    email:'sofia@mail.com',    titular:'Diseñadora UX',
ciudad:'Medellín',     anios:3},
  {id:'u6',  nombre:'Diego Ramírez',   email:'diego@mail.com',    titular:'Desarrollador
Frontend',    ciudad:'Cali',         anios:2},
  {id:'u7',  nombre:'Laura Mejía',     email:'laura@mail.com',    titular:'Analista de
Seguridad',     ciudad:'Cali',         anios:5},
  {id:'u8',  nombre:'Andrés Castro',   email:'andres@mail.com',   titular:'Arquitecto de
Software',    ciudad:'Bogotá',       anios:10},
  {id:'u9',  nombre:'Paula Herrera',   email:'paula@mail.com',    titular:'Product Owner',
ciudad:'Bogotá',       anios:6},
  {id:'u10', nombre:'Julián Ortiz',    email:'julian@mail.com',   titular:'Ingeniero ML',
ciudad:'Medellín',     anios:4},
  {id:'u11', nombre:'Camila Duarte',   email:'camila@mail.com',   titular:'Analista BI',
ciudad:'Barranquilla', anios:3},
  {id:'u12', nombre:'Felipe Salazar',  email:'felipe@mail.com',   titular:'Desarrollador Móvil',
ciudad:'Cali',         anios:3},
  {id:'u13', nombre:'Valentina Cruz',  email:'valentina@mail.com',titular:'Ingeniera Cloud',
ciudad:'Bogotá',       anios:5},
  {id:'u14', nombre:'Mateo Núñez',     email:'mateo@mail.com',    titular:'Desarrollador Full
Stack',  ciudad:'Medellín',     anios:4},
  {id:'u15', nombre:'Isabela León',    email:'isabela@mail.com',  titular:'Scrum Master',
ciudad:'Bogotá',       anios:8},
  {id:'u16', nombre:'Sebastián Mora',  email:'sebastian@mail.com',titular:'Ingeniero de Datos',
ciudad:'Barranquilla', anios:2},
  {id:'u17', nombre:'Daniela Pardo',   email:'daniela@mail.com',  titular:'Investigadora en IA',
ciudad:'Bogotá',       anios:6},
```

```
  {id:'u18', nombre:'Nicolás Rojas',   email:'nicolas@mail.com',  titular:'Desarrollador
Backend',     ciudad:'Cali',         anios:1},
  {id:'u19', nombre:'Gabriela Soto',   email:'gabriela@mail.com', titular:'Diseñadora de
Producto',    ciudad:'Medellín',     anios:4},
  {id:'u20', nombre:'Tomás Aguirre',   email:'tomas@mail.com',    titular:'Analista de Datos',
ciudad:'Barranquilla', anios:2}
] AS r
MERGE (u:Usuario {id:r.id})
SET u.nombre=r.nombre, u.email=r.email, u.titular=r.titular,
    u.ciudad=r.ciudad, u.anios_experiencia=r.anios;

// ---------- Proyectos ----------
UNWIND [
  {id:'p1', nombre:'Plataforma de Analítica',   descripcion:'Dashboards en tiempo real'},
  {id:'p2', nombre:'Motor de Recomendación',    descripcion:'Sistema de recomendación con ML'},
  {id:'p3', nombre:'App Móvil Bancaria',        descripcion:'Aplicación móvil de pagos'},
  {id:'p4', nombre:'Migración a la Nube',       descripcion:'Migración de servicios a AWS'},
  {id:'p5', nombre:'Portal de Empleo',          descripcion:'Portal web de vacantes'},
  {id:'p6', nombre:'Sistema Antifraude',        descripcion:'Detección de fraude'},
  {id:'p7', nombre:'Rediseño de Marca Digital', descripcion:'Nueva experiencia web'},
  {id:'p8', nombre:'Monitoreo de Cultivos',     descripcion:'Sensores y analítica agrícola'}
] AS r
MERGE (p:Proyecto {id:r.id})
SET p.nombre=r.nombre, p.descripcion=r.descripcion;

// ---------- Conexiones entre usuarios (se guardan en un sentido, se consultan sin dirección) --
--------
UNWIND [
  ['u1','u2'],['u1','u3'],['u1','u4'],['u1','u9'],
  ['u2','u4'],['u2','u8'],['u2','u14'],
  ['u3','u10'],['u3','u17'],['u3','u11'],
  ['u4','u13'],['u4','u8'],
  ['u5','u19'],['u5','u6'],['u5','u14'],
  ['u6','u12'],['u6','u14'],
  ['u7','u12'],['u7','u18'],['u7','u4'],
  ['u8','u9'],['u8','u13'],['u8','u15'],
  ['u9','u15'],
  ['u10','u17'],['u10','u1'],
  ['u11','u16'],['u11','u20'],
  ['u16','u20'],['u16','u1'],
  ['u12','u18'],
  ['u13','u15'],
  ['u19','u9']
] AS par
MATCH (a:Usuario {id:par[0]}), (b:Usuario {id:par[1]})
MERGE (a)-[c:CONECTA_CON]->(b)
  ON CREATE SET c.desde = date('2024-01-01');

// ---------- Usuario trabaja en empresa ----------
UNWIND [
  ['u1','e2','Ingeniera de Datos'],   ['u2','e1','Backend Developer'],
  ['u3','e2','Científica de Datos'],  ['u4','e3','DevOps'],
  ['u5','e5','Diseñadora UX'],        ['u6','e5','Frontend Developer'],
  ['u7','e4','Analista de Seguridad'],['u8','e1','Arquitecto'],
  ['u9','e1','Product Owner'],        ['u10','e2','Ingeniero ML'],
  ['u11','e6','Analista BI'],         ['u12','e4','Desarrollador Móvil'],
  ['u13','e3','Ingeniera Cloud'],     ['u14','e5','Full Stack'],
  ['u15','e1','Scrum Master'],        ['u16','e6','Ingeniero de Datos'],
  ['u17','e2','Investigadora IA'],    ['u18','e4','Backend Developer'],
  ['u19','e5','Diseñadora'],          ['u20','e6','Analista de Datos']
] AS r
MATCH (u:Usuario {id:r[0]}), (e:Empresa {id:r[1]})
MERGE (u)-[t:TRABAJA_EN]->(e)
  ON CREATE SET t.cargo = r[2], t.desde = date('2022-01-01');
```

```
// ---------- Usuario tiene habilidad (nivel 1-5) ----------
UNWIND [
  ['u1','Python',5],['u1','SQL',5],['u1','Data Analysis',4],['u1','Neo4j',3],
  ['u2','Java',5],['u2','SQL',4],['u2','Docker',3],['u2','Node.js',3],
  ['u3','Python',5],['u3','Machine Learning',5],['u3','Data Analysis',4],
  ['u4','Docker',5],['u4','AWS',5],['u4','Python',3],
  ['u5','UX Design',5],['u5','JavaScript',2],
  ['u6','JavaScript',4],['u6','React',4],['u6','UX Design',3],
  ['u7','Cybersecurity',5],['u7','Python',3],['u7','SQL',3],
  ['u8','Java',5],['u8','AWS',4],['u8','Docker',4],['u8','SQL',4],['u8','Neo4j',3],
  ['u9','Scrum',4],['u9','Data Analysis',3],
  ['u10','Python',4],['u10','Machine Learning',5],['u10','Docker',3],
  ['u11','Power BI',5],['u11','SQL',4],['u11','Data Analysis',4],
  ['u12','Kotlin',5],['u12','Java',4],
  ['u13','AWS',5],['u13','Docker',4],['u13','Python',3],
  ['u14','JavaScript',5],['u14','React',4],['u14','Node.js',4],['u14','SQL',3],
  ['u15','Scrum',5],
  ['u16','Python',3],['u16','SQL',4],['u16','Neo4j',2],['u16','Data Analysis',3],
  ['u17','Machine Learning',5],['u17','Python',5],
  ['u18','Java',3],['u18','Node.js',2],['u18','SQL',3],
  ['u19','UX Design',4],['u19','Scrum',2],
  ['u20','Power BI',3],['u20','SQL',3],['u20','Data Analysis',3]
] AS r
MATCH (u:Usuario {id:r[0]}), (h:Habilidad {nombre:r[1]})
MERGE (u)-[t:TIENE_HABILIDAD]->(h)
  ON CREATE SET t.nivel = r[2];

// ---------- Usuario participa en proyecto ----------
UNWIND [
  ['u1','p1'],['u3','p1'],['u11','p1'],['u16','p1'],
  ['u3','p2'],['u10','p2'],['u17','p2'],
  ['u12','p3'],['u7','p3'],['u18','p3'],
  ['u4','p4'],['u13','p4'],['u8','p4'],
  ['u2','p5'],['u14','p5'],['u9','p5'],['u6','p5'],
  ['u7','p6'],['u1','p6'],
  ['u5','p7'],['u19','p7'],['u6','p7'],
  ['u11','p8'],['u20','p8'],['u16','p8']
] AS r
MATCH (u:Usuario {id:r[0]}), (p:Proyecto {id:r[1]})
MERGE (u)-[:PARTICIPA_EN]->(p);

// ---------- Tecnologías usadas por proyecto ----------
UNWIND [
  ['p1','Python'],['p1','SQL'],['p1','Power BI'],['p1','Data Analysis'],
  ['p2','Python'],['p2','Machine Learning'],['p2','Neo4j'],
  ['p3','Kotlin'],['p3','Java'],['p3','Cybersecurity'],
  ['p4','AWS'],['p4','Docker'],
  ['p5','React'],['p5','Node.js'],['p5','SQL'],['p5','JavaScript'],
  ['p6','Python'],['p6','Machine Learning'],['p6','Cybersecurity'],
  ['p7','UX Design'],['p7','React'],
  ['p8','Python'],['p8','Data Analysis'],['p8','Power BI']
] AS r
MATCH (p:Proyecto {id:r[0]}), (h:Habilidad {nombre:r[1]})
MERGE (p)-[:USA_TECNOLOGIA]->(h);

// ---------- Ofertas laborales (publicadas por empresas) ----------
UNWIND [
  {id:'o1', empresa:'e1', titulo:'Desarrollador Backend Senior', ubicacion:'Bogotá',
salario:9000000,  estado:'ABIERTA'},
  {id:'o2', empresa:'e2', titulo:'Científico de Datos',          ubicacion:'Medellín',
salario:10000000, estado:'ABIERTA'},
```

```
  {id:'o3', empresa:'e3', titulo:'Ingeniero DevOps',            ubicacion:'Bogotá',
salario:11000000, estado:'ABIERTA'},
  {id:'o4', empresa:'e4', titulo:'Analista de Ciberseguridad',   ubicacion:'Cali',
salario:8500000,  estado:'ABIERTA'},
  {id:'o5', empresa:'e5', titulo:'Diseñador UX/UI',              ubicacion:'Medellín',
salario:7000000,  estado:'ABIERTA'},
  {id:'o6', empresa:'e6', titulo:'Analista BI',                  ubicacion:'Barranquilla',
salario:6500000,  estado:'ABIERTA'},
  {id:'o7', empresa:'e1', titulo:'Desarrollador Full Stack',     ubicacion:'Bogotá',
salario:8000000,  estado:'ABIERTA'},
  {id:'o8', empresa:'e2', titulo:'Ingeniero de Datos',           ubicacion:'Medellín',
salario:9500000,  estado:'ABIERTA'},
  {id:'o9', empresa:'e3', titulo:'Arquitecto Cloud',             ubicacion:'Bogotá',
salario:13000000, estado:'ABIERTA'},
  {id:'o10',empresa:'e4', titulo:'Desarrollador Móvil Kotlin',   ubicacion:'Cali',
salario:8000000,  estado:'CERRADA'}
] AS r
MATCH (e:Empresa {id:r.empresa})
MERGE (o:Oferta {id:r.id})
SET o.titulo=r.titulo, o.ubicacion=r.ubicacion, o.salario=r.salario, o.estado=r.estado
MERGE (e)-[:PUBLICA]->(o);

// ---------- Habilidades requeridas por oferta ----------
UNWIND [
  ['o1','Java'],['o1','SQL'],['o1','Docker'],
  ['o2','Python'],['o2','Machine Learning'],['o2','Data Analysis'],
  ['o3','Docker'],['o3','AWS'],['o3','Python'],
  ['o4','Cybersecurity'],['o4','Python'],['o4','SQL'],
  ['o5','UX Design'],['o5','JavaScript'],
  ['o6','Power BI'],['o6','SQL'],['o6','Data Analysis'],
  ['o7','JavaScript'],['o7','React'],['o7','Node.js'],['o7','SQL'],
  ['o8','Python'],['o8','SQL'],['o8','Neo4j'],
  ['o9','AWS'],['o9','Docker'],['o9','Java'],
  ['o10','Kotlin'],['o10','Java']
] AS r
MATCH (o:Oferta {id:r[0]}), (h:Habilidad {nombre:r[1]})
MERGE (o)-[:REQUIERE]->(h);

// ---------- Postulaciones ----------
UNWIND [
  ['u2','o1','2026-10-01','EN_REVISION'],['u8','o1','2026-10-02','EN_REVISION'],
  ['u3','o2','2026-10-03','EN_REVISION'],['u17','o2','2026-10-04','ENTREVISTA'],
  ['u10','o2','2026-10-05','EN_REVISION'],
  ['u4','o3','2026-10-06','EN_REVISION'],['u13','o3','2026-10-06','ENTREVISTA'],
  ['u7','o4','2026-10-07','EN_REVISION'],
  ['u5','o5','2026-10-08','EN_REVISION'],['u19','o5','2026-10-09','EN_REVISION'],
  ['u11','o6','2026-10-10','EN_REVISION'],['u20','o6','2026-10-11','EN_REVISION'],
  ['u14','o7','2026-10-12','EN_REVISION'],['u6','o7','2026-10-12','EN_REVISION'],
  ['u1','o8','2026-10-13','ENTREVISTA'],['u16','o8','2026-10-14','EN_REVISION'],
  ['u13','o9','2026-10-15','EN_REVISION'],['u8','o9','2026-10-15','EN_REVISION']
] AS r
MATCH (u:Usuario {id:r[0]}), (o:Oferta {id:r[1]})
MERGE (u)-[p:POSTULA_A]->(o)
  ON CREATE SET p.fecha = date(r[2]), p.estado = r[3];
```

Anexo C. 03_queries.cypher

```
// ======================================================================
// 03_queries.cypher  -  Consultas del proyecto (Neo4j 5.x)
// Cada consulta usa valores de ejemplo; en la app se pasan como parámetros.
// Ejecutar de a una en Neo4j Browser (http://localhost:7474).
// ======================================================================



// ----------------------------------------------------------------------
// A. CRUD
// ----------------------------------------------------------------------

// A1. Crear usuario
CREATE (u:Usuario {id:'u21', nombre:'Elena Prado', email:'elena@mail.com',
                   titular:'Ingeniera de Datos', ciudad:'Bogotá', anios_experiencia:3})
RETURN u;

// A2. Consultar usuario con su perfil completo
MATCH (u:Usuario {id:'u1'})
OPTIONAL MATCH (u)-[:TRABAJA_EN]->(e:Empresa)
OPTIONAL MATCH (u)-[th:TIENE_HABILIDAD]->(h:Habilidad)
RETURN u, e.nombre AS empresa, collect({habilidad:h.nombre, nivel:th.nivel}) AS habilidades;

// A3. Actualizar usuario
MATCH (u:Usuario {id:'u21'})
SET u.titular = 'Científica de Datos Senior', u.anios_experiencia = 4
RETURN u;

// A4. Eliminar usuario y todas sus relaciones
MATCH (u:Usuario {id:'u21'})
DETACH DELETE u;



// ----------------------------------------------------------------------
// B. Relaciones
// ----------------------------------------------------------------------

// B1. Conectar dos usuarios (MERGE evita duplicados aunque se invierta el orden)
MATCH (a:Usuario {id:'u5'}), (b:Usuario {id:'u9'})
WHERE NOT (a)-[:CONECTA_CON]-(b)
CREATE (a)-[:CONECTA_CON {desde: date()}]->(b);

// B2. Asignar habilidad a usuario
MATCH (u:Usuario {id:'u5'}), (h:Habilidad {nombre:'React'})
MERGE (u)-[t:TIENE_HABILIDAD]->(h) SET t.nivel = 3;

// B3. Vincular usuario con empresa
MATCH (u:Usuario {id:'u18'}), (e:Empresa {id:'e1'})
MERGE (u)-[t:TRABAJA_EN]->(e) SET t.cargo = 'Backend Developer', t.desde = date();

// B4. Vincular usuario con proyecto
MATCH (u:Usuario {id:'u5'}), (p:Proyecto {id:'p5'})
MERGE (u)-[:PARTICIPA_EN]->(p);

// B5. Empresa publica oferta y la ligamos con las habilidades requeridas
MATCH (e:Empresa {id:'e3'})
CREATE (o:Oferta {id:'o11', titulo:'Ingeniero SRE', ubicacion:'Bogotá',
                  salario:12000000, estado:'ABIERTA'})
```

```
CREATE (e)-[:PUBLICA]->(o)
WITH o
UNWIND ['Docker','AWS','Python'] AS hab
MATCH (h:Habilidad {nombre:hab})
CREATE (o)-[:REQUIERE]->(h)
RETURN o;

// B6. Postulación de un usuario a una oferta
MATCH (u:Usuario {id:'u4'}), (o:Oferta {id:'o9'})
MERGE (u)-[p:POSTULA_A]->(o)
  ON CREATE SET p.fecha = date(), p.estado = 'EN_REVISION'
RETURN u.nombre, o.titulo, p.estado;



// ---------------------------------------------------------------------
// C. Preguntas del enunciado
// ---------------------------------------------------------------------

// C1. ¿Qué usuarios tienen habilidades similares? (índice de Jaccard)
MATCH (a:Usuario {id:'u1'})-[:TIENE_HABILIDAD]->(h:Habilidad)<-[:TIENE_HABILIDAD]-(b:Usuario)
WHERE a <> b
WITH a, b, collect(h.nombre) AS compartidas
MATCH (a)-[:TIENE_HABILIDAD]->(ha:Habilidad)
WITH a, b, compartidas, collect(ha) AS habA
MATCH (b)-[:TIENE_HABILIDAD]->(hb:Habilidad)
WITH a, b, compartidas, habA, collect(hb) AS habB
RETURN b.nombre AS usuario, compartidas,
       size(compartidas) * 1.0 /
         (size(habA) + size(habB) - size(compartidas)) AS similitud
ORDER BY similitud DESC, usuario
LIMIT 10;

// C2. ¿Qué contactos en común existen entre dos profesionales?
MATCH (a:Usuario {id:'u1'})-[:CONECTA_CON]-(comun:Usuario)-[:CONECTA_CON]-(b:Usuario {id:'u8'})
RETURN comun.nombre AS contacto_en_comun, comun.titular AS titular;

// C3. ¿Cuál es la ruta más corta entre un usuario y una empresa?
//     (solo relaciones del modelo profesional: no pasa por invitaciones, publicaciones ni
mensajes; máx. 8 saltos)
MATCH (u:Usuario {id:'u6'}), (e:Empresa {nombre:'CloudNine'}),
      p = shortestPath((u)-
[:CONECTA_CON|TRABAJA_EN|PARTICIPA_EN|POSTULA_A|PUBLICA|REQUIERE|TIENE_HABILIDAD|USA_TECNOLOGIA*.
.8]-(e))
RETURN p, length(p) AS saltos,
       [n IN nodes(p) | coalesce(n.nombre, n.titulo)] AS camino;

// C3b. Variante restringida a contactos profesionales + trabajo (más "humana")
MATCH (u:Usuario {id:'u6'}), (e:Empresa {nombre:'CloudNine'}),
      p = shortestPath((u)-[:CONECTA_CON|TRABAJA_EN*..8]-(e))
RETURN [n IN nodes(p) | coalesce(n.nombre, n.titulo)] AS camino, length(p) AS saltos;

// C4. ¿Qué candidatos podrían recomendarse para una oferta según sus conexiones y competencias?
//     puntaje = 70 % cobertura de habilidades + 30 % conexiones con empleados de la empresa
MATCH (o:Oferta {id:'o3'})-[:REQUIERE]->(req:Habilidad)
WITH o, collect(req) AS requeridas
MATCH (o)<-[:PUBLICA]-(emp:Empresa)
MATCH (u:Usuario)
WHERE NOT (u)-[:TRABAJA_EN]->(emp) AND NOT (u)-[:POSTULA_A]->(o)
WITH o, emp, requeridas, u,
     [h IN requeridas WHERE (u)-[:TIENE_HABILIDAD]->(h) | h.nombre] AS cumple
WHERE size(cumple) > 0
OPTIONAL MATCH (u)-[:CONECTA_CON]-(colega:Usuario)-[:TRABAJA_EN]->(emp)
```

```
WITH u, requeridas, cumple, count(DISTINCT colega) AS conexiones_empresa
RETURN u.nombre AS candidato, cumple AS habilidades_cumplidas, conexiones_empresa,
       round(100 * (0.7 * size(cumple) / size(requeridas)
                    + 0.3 * toFloat(CASE WHEN conexiones_empresa > 3 THEN 3 ELSE
conexiones_empresa END) / 3)) AS puntaje
ORDER BY puntaje DESC
LIMIT 10;

// C5. ¿Qué proyectos comparten tecnologías o miembros relacionados?
MATCH (p1:Proyecto)-[:USA_TECNOLOGIA]->(h:Habilidad)<-[:USA_TECNOLOGIA]-(p2:Proyecto)
WHERE p1.id < p2.id
OPTIONAL MATCH (p1)<-[:PARTICIPA_EN]-(m:Usuario)-[:PARTICIPA_EN]->(p2)
RETURN p1.nombre AS proyecto_1, p2.nombre AS proyecto_2,
       collect(DISTINCT h.nombre) AS tecnologias_comunes,
       collect(DISTINCT m.nombre) AS miembros_comunes
ORDER BY size(collect(DISTINCT h.nombre)) DESC
LIMIT 10;

// C6. ¿Qué habilidades son las más demandadas en las ofertas?
MATCH (o:Oferta {estado:'ABIERTA'})-[:REQUIERE]->(h:Habilidad)
RETURN h.nombre AS habilidad, count(o) AS ofertas
ORDER BY ofertas DESC, habilidad
LIMIT 10;

// C6b. Brecha: demanda vs. oferta de talento
MATCH (h:Habilidad)
OPTIONAL MATCH (o:Oferta {estado:'ABIERTA'})-[:REQUIERE]->(h)
WITH h, count(DISTINCT o) AS demanda
OPTIONAL MATCH (u:Usuario)-[:TIENE_HABILIDAD]->(h)
RETURN h.nombre AS habilidad, demanda, count(DISTINCT u) AS profesionales,
       demanda - count(DISTINCT u) AS brecha
ORDER BY brecha DESC;



// ------------------------------------------------------------------------
// D. Recomendaciones
// ------------------------------------------------------------------------

// D1. Contactos sugeridos: amigos de amigos (aún no conectados)
MATCH (yo:Usuario {id:'u1'})-[:CONECTA_CON]-(:Usuario)-[:CONECTA_CON]-(sug:Usuario)
WHERE sug <> yo AND NOT (yo)-[:CONECTA_CON]-(sug)
RETURN sug.nombre AS sugerido, sug.titular AS titular, count(*) AS contactos_comunes
ORDER BY contactos_comunes DESC
LIMIT 5;

// D2. Empleos sugeridos: ofertas abiertas con más habilidades del usuario
MATCH (yo:Usuario {id:'u14'})
MATCH (o:Oferta {estado:'ABIERTA'})-[:REQUIERE]->(h:Habilidad)
WHERE NOT (yo)-[:POSTULA_A]->(o)
WITH yo, o, collect(h) AS requeridas
WITH yo, o, requeridas,
     [x IN requeridas WHERE (yo)-[:TIENE_HABILIDAD]->(x) | x.nombre] AS cumple
WHERE size(cumple) > 0
MATCH (o)<-[:PUBLICA]-(emp:Empresa)
RETURN o.titulo AS oferta, emp.nombre AS empresa, cumple AS coincidencias,
       size(cumple) AS n, size(requeridas) AS total,
       round(100.0 * size(cumple) / size(requeridas)) AS porcentaje
ORDER BY porcentaje DESC, n DESC
LIMIT 5;

// D3. Colaboradores sugeridos: personas con habilidades en proyectos afines a los míos
MATCH (yo:Usuario {id:'u1'})-[:PARTICIPA_EN]->(:Proyecto)-[:USA_TECNOLOGIA]->(h:Habilidad)
```

```
      <-[:USA_TECNOLOGIA]-(p2:Proyecto)<-[:PARTICIPA_EN]-(col:Usuario)
WHERE col <> yo AND NOT (yo)-[:PARTICIPA_EN]->(p2)
RETURN col.nombre AS colaborador, collect(DISTINCT p2.nombre) AS proyectos,
       count(DISTINCT h) AS tecnologias_afines
ORDER BY tecnologias_afines DESC
LIMIT 5;



// ------------------------------------------------------------------------
// E. Analítica
// ------------------------------------------------------------------------

// E1. Usuarios más influyentes (centralidad de grado)
MATCH (u:Usuario)
OPTIONAL MATCH (u)-[:CONECTA_CON]-(c:Usuario)
RETURN u.nombre AS usuario, count(DISTINCT c) AS conexiones
ORDER BY conexiones DESC
LIMIT 10;

// E2. Empresas con más postulaciones
MATCH (e:Empresa)-[:PUBLICA]->(o:Oferta)<-[:POSTULA_A]-(u:Usuario)
RETURN e.nombre AS empresa, count(u) AS postulaciones
ORDER BY postulaciones DESC;

// E3. Comunidades con Graph Data Science (OPCIONAL: requiere el plugin GDS)
//     La aplicación calcula comunidades en Python (networkx) para no depender del plugin.
// CALL gds.graph.project('red', 'Usuario', {CONECTA_CON:{orientation:'UNDIRECTED'}});
// CALL gds.louvain.stream('red')
// YIELD nodeId, communityId
// RETURN communityId, collect(gds.util.asNode(nodeId).nombre) AS miembros
// ORDER BY size(miembros) DESC;
// CALL gds.pageRank.stream('red') YIELD nodeId, score
// RETURN gds.util.asNode(nodeId).nombre AS usuario, score ORDER BY score DESC LIMIT 10;
```

Anexo D. 04_red_social.cypher

```
// ================================================================
// 04_red_social.cypher  -  Datos de ejemplo de UniRed (la capa social)
// Instituciones y estudios, publicaciones, reacciones, comentarios,
// invitaciones de conexión y algunos mensajes recientes.
// Idempotente (MERGE por id). Las fechas son relativas a "ahora".
// ================================================================

// ---------- Instituciones ----------
UNWIND ['Universidad del Norte','Universidad de los Andes','Universidad Nacional de Colombia',
        'Universidad EAFIT','Universidad del Valle','Pontificia Universidad Javeriana','SENA'] AS
n
MERGE (:Institucion {nombre: n});

// ---------- Estudios: (Usuario)-[:ESTUDIO_EN]->(Institucion) ----------
UNWIND [
  ['u1','Universidad Nacional de Colombia','Ingeniería de Sistemas','Datos','2012','2017'],
  ['u2','Universidad de los Andes','Ingeniería de Software','Backend','2015','2020'],
  ['u3','Universidad EAFIT','Matemáticas Aplicadas','Estadística','2013','2018'],
  ['u4','Universidad Nacional de Colombia','Ingeniería Electrónica','Redes','2010','2015'],
  ['u5','Universidad EAFIT','Diseño Interactivo','UX','2016','2021'],
  ['u6','Universidad del Valle','Tecnología en Desarrollo de Software','Frontend','2019','2022'],
  ['u7','Universidad del Valle','Ingeniería de Sistemas','Seguridad','2013','2018'],
  ['u8','Universidad de los Andes','Ingeniería de Sistemas','Arquitectura','2008','2013'],
  ['u8','Universidad de los Andes','Maestría en Ingeniería de Software','','2016','2018'],
  ['u9','Pontificia Universidad Javeriana','Administración de Empresas','Gestión de
producto','2012','2017'],
  ['u10','Universidad EAFIT','Ingeniería Matemática','Machine Learning','2015','2020'],
  ['u11','Universidad del Norte','Ingeniería Industrial','Analítica','2017','2022'],
  ['u12','Universidad del Valle','Ingeniería de Sistemas','Móvil','2016','2021'],
  ['u13','Universidad Nacional de Colombia','Ingeniería de Sistemas','Nube','2014','2019'],
  ['u14','Universidad EAFIT','Ingeniería de Sistemas','','2016','2021'],
  ['u15','Pontificia Universidad Javeriana','Ingeniería Industrial','Agilidad','2010','2015'],
  ['u16','Universidad del Norte','Ingeniería de Sistemas','Bases de datos','2020','2025'],
  ['u17','Universidad de los Andes','Doctorado en Ingeniería','Inteligencia
artificial','2016','2021'],
  ['u18','SENA','Tecnólogo en Análisis y Desarrollo de Software','','2022','2024'],
  ['u19','Universidad EAFIT','Diseño Industrial','Producto digital','2016','2021'],
  ['u20','Universidad del Norte','Ingeniería de Sistemas','Ciencia de datos','2021','']
] AS r
MATCH (u:Usuario {id:r[0]}), (i:Institucion {nombre:r[1]})
MERGE (u)-[s:ESTUDIO_EN {id: 's-' + r[0] + '-' + r[4]}]->(i)
  ON CREATE SET s.titulo = r[2], s.area = r[3], s.inicio = r[4], s.fin = r[5];

// ---------- Textos de perfil ----------
UNWIND [
  ['u1','Diseño pipelines de datos y modelos de grafos. Me interesa conectar a quienes empiezan
en datos con proyectos reales.'],
  ['u8','Arquitecto de software con 10 años de experiencia en sistemas distribuidos. Mentor de
estudiantes de último semestre.'],
  ['u11','Analista BI en Barranquilla. Power BI, SQL y mucha curiosidad por la logística del
Caribe.'],
  ['u16','Egresado de Uninorte. Trabajo con Neo4j y SQL, y me gusta explicar bases de datos con
ejemplos sencillos.'],
  ['u20','Estudiante de Ingeniería de Sistemas en Uninorte. Buscando mi primera práctica en
análisis de datos.']
] AS r
MATCH (u:Usuario {id:r[0]}) SET u.acerca = r[1];

// ---------- Publicaciones: (Usuario)-[:PUBLICO]->(Publicacion) ----------
UNWIND [
```

```
  ['p1','u16', 3,   'Terminamos la migración del modelo de recomendaciones a Neo4j. La consulta
de "amigos de amigos" pasó de 4 JOIN a una línea de Cypher. Si alguien de Uninorte quiere ver el
modelo, escríbame.'],
  ['p2','u8',  9,   'Consejo para quien está por graduarse: muestren un proyecto terminado,
aunque sea pequeño, antes que diez a medias. Se nota la diferencia en una entrevista.'],
  ['p3','u20', 20,  '¡Busco práctica profesional en análisis de datos para el próximo semestre!
Manejo Python, SQL y Power BI. Cualquier recomendación es bienvenida.'],
  ['p4','u13', 27,  'Nuevo artículo en el blog del equipo: cómo bajamos 40 % la factura de AWS
apagando entornos de prueba por la noche. Spoiler: fue un script de 30 líneas.'],
  ['p5','u5',  35,  'En PixelWorks estamos haciendo pruebas de usabilidad con estudiantes. Si
quieres participar (45 min, virtual) comenta abajo.'],
  ['p6','u11', 50,  'Primer tablero publicado en AgroSmart: precios de cosecha por municipio del
Atlántico. Orgullosa del equipo.'],
  ['p7','u10', 72,  'Comparto el material del taller de Machine Learning que dimos en Medellín.
Incluye los cuadernos y el conjunto de datos.'],
  ['p8','u7',  96,  'Recordatorio: la autenticación de dos factores no es opcional. Esta semana
vimos tres cuentas comprometidas por reutilizar contraseñas.'],
  ['p9','u15', 130, 'Retro del sprint: menos reuniones, más tablero. Pasamos de 6 h a 2 h
semanales de ceremonias y la velocidad del equipo subió.'],
  ['p10','u3', 170, 'Hoy cumplo 5 años como científica de datos. Gracias a todos los que me
ayudaron a cambiar de las matemáticas puras a la industria.']
] AS r
MATCH (a:Usuario {id:r[1]})
MERGE (p:Publicacion {id:r[0]})
  ON CREATE SET p.texto = r[3], p.fecha = datetime() - duration({hours: r[2]})
MERGE (a)-[:PUBLICO]->(p);

// ---------- Reacciones: (Usuario)-[:REACCIONO {tipo}]->(Publicacion) ----------
UNWIND [

['u1','p1','recomendar'],['u20','p1','celebrar'],['u11','p1','interesante'],['u8','p1','recomenda
r'],['u2','p1','encanta'],

['u16','p2','recomendar'],['u20','p2','apoyar'],['u14','p2','recomendar'],['u9','p2','interesante
'],
  ['u16','p3','apoyar'],['u11','p3','apoyar'],['u1','p3','recomendar'],
  ['u4','p4','celebrar'],['u8','p4','interesante'],
  ['u19','p5','encanta'],['u6','p5','recomendar'],
  ['u16','p6','celebrar'],['u20','p6','celebrar'],['u1','p6','encanta'],
  ['u17','p7','recomendar'],['u3','p7','encanta'],
  ['u12','p8','recomendar'],['u4','p8','apoyar'],
  ['u9','p9','divertido'],['u2','p9','recomendar'],
  ['u10','p10','celebrar'],['u17','p10','celebrar'],['u1','p10','encanta']
] AS r
MATCH (u:Usuario {id:r[0]}), (p:Publicacion {id:r[1]})
MERGE (u)-[x:REACCIONO]->(p)
  ON CREATE SET x.tipo = r[2], x.fecha = p.fecha + duration({minutes: 20});

// ---------- Comentarios: (Usuario)-[:ESCRIBIO]->(Comentario)-[:SOBRE]->(Publicacion) ----------
UNWIND [
  ['k1','u20','p1','¡Me interesa mucho! Estoy viendo grafos en Bases de Datos este semestre.',
1],
  ['k2','u1','p1','Muy buen resultado. ¿Usaron restricciones de unicidad desde el inicio?', 2],
  ['k3','u16','p3','Te escribo por mensaje, en mi equipo abren prácticas en enero.', 18],
  ['k4','u11','p3','Revisa también las ofertas de AgroSmart, buscan perfiles de datos.', 15],
  ['k5','u19','p5','¡Cuenta conmigo!', 30],
  ['k6','u14','p2','Totalmente de acuerdo, el portafolio pesa más que el promedio.', 7]
] AS r
MATCH (u:Usuario {id:r[1]}), (p:Publicacion {id:r[2]})
MERGE (k:Comentario {id:r[0]})
  ON CREATE SET k.texto = r[3], k.fecha = datetime() - duration({hours: r[4]})
MERGE (u)-[:ESCRIBIO]->(k)
MERGE (k)-[:SOBRE]->(p);
```

```
// ---------- Invitaciones pendientes: (Usuario)-[:SOLICITA_CONEXION]->(Usuario) ----------
UNWIND [['u20','u8'],['u18','u16'],['u6','u16'],['u17','u20']] AS par
MATCH (a:Usuario {id:par[0]}), (b:Usuario {id:par[1]})
WHERE NOT (a)-[:CONECTA_CON]-(b)
MERGE (a)-[s:SOLICITA_CONEXION]->(b) ON CREATE SET s.fecha = datetime() - duration({hours: 5});

// ---------- Algunas conexiones nuevas entre egresados de Uninorte ----------
UNWIND [['u16','u20'],['u1','u16'],['u11','u20'],['u11','u16']] AS par
MATCH (a:Usuario {id:par[0]}), (b:Usuario {id:par[1]})
WHERE NOT (a)-[:CONECTA_CON]-(b)
CREATE (a)-[:CONECTA_CON {desde: date()}]->(b);

// ---------- Mensajes (se borran solos a las 24 h) ----------
UNWIND [
  ['m1','u16','u20','Hola Tomás, vi tu publicación. ¿Me mandas tu hoja de vida?', 180],
  ['m2','u20','u16','¡Claro! Te la envío hoy mismo. Muchas gracias.', 170],
  ['m3','u16','u20','Perfecto, la paso al líder del equipo de datos.', 160],
  ['m4','u1','u16','¿Nos vemos el jueves para revisar el modelo del grafo?', 45]
] AS r
MATCH (a:Usuario {id:r[1]}), (b:Usuario {id:r[2]})
WHERE (a)-[:CONECTA_CON]-(b)
MERGE (m:Mensaje {id:r[0]})
  ON CREATE SET m.texto = r[3], m.fecha = datetime() - duration({minutes: r[4]}), m.leido = false
MERGE (a)-[:ENVIO]->(m)
MERGE (m)-[:PARA]->(b);
```